PowerFlowNet: Leveraging Message Passing GNNs for Improved Power Flow Approximation

Nan Lin *Student Member, IEEE*, Stavros Orfanoudakis *Student Member, IEEE*, Nathan Ordonez Cardenas, Juan S. Giraldo, *Member, IEEE*, Pedro P. Vergara, *Senior Member, IEEE*

arXiv:2311.03415v1 [cs.LG] 6 Nov 2023

***Abstract*—Accurate and efficient power flow (PF) analysis is crucial in modern electrical networks' efficient operation and planning. Therefore, there is a need for scalable algorithms capable of handling large-scale power networks that can provide accurate and fast solutions. Graph Neural Networks (GNNs) have emerged as a promising approach for enhancing the speed of PF approximations by leveraging their ability to capture distinctive features from the underlying power network graph. In this study, we introduce PowerFlowNet, a novel GNN architecture for PF approximation that showcases similar performance with the traditional Newton-Raphson method but achieves it 4 times faster in the simple IEEE 14-bus system and 145 times faster in the realistic case of the French high voltage network (6470rte). Meanwhile, it significantly outperforms other traditional approximation methods, such as the DC relaxation method, in terms of performance and execution time; therefore, making PowerFlowNet a highly promising solution for real-world PF analysis. Furthermore, we verify the efficacy of our approach by conducting an in-depth experimental evaluation, thoroughly examining the performance, scalability, interpretability, and architectural dependability of PowerFlowNet. The evaluation provides insights into the behavior and potential applications of GNNs in power system analysis.**

***Index Terms*—Power Flow State Estimation, Deep Learning, Graph Neural Network.**

I. Introduction

THE complexity of electrical power systems is continuously rising, largely attributed to the substantial integration of decentralized renewable energy resources. Within this context, power flow (PF) stands as a fundamental challenge in ensuring the stability of power systems, playing a pivotal role in both the operational management and long-term planning of electrical networks. At its core, PF is a mathematical problem that revolves around determining the voltages at various buses, a task accomplished by solving a set of nonlinear equations, which are inherently linked to the network configuration, load distribution, and generation characteristics [1]. Traditional methods, such as Newton-Raphson [2], the Gauss-Seidel [3], and the fast-decoupled methods, have excellent accuracy and convergence properties but scale slowly with larger power systems, particularly in long-term planning of national grids with thousands of buses. In contrast, the DC power flow (DCPF) technique [4] simplifies this problem into a linear one by making assumptions about the voltage magnitude, but at the cost of accuracy. Consequently, there is a pressing requirement for innovative algorithms capable of efficiently solving the PF for significantly larger networks. These novel algorithms should aim to balance the need for speed without compromising accuracy, ultimately facilitating the streamlined operation and planning of large real-world electrical grids.

Modernizing the grids with the addition of accurate smart metering and data acquisition systems has enabled the development of Machine Learning (ML) methods for accurate and efficient power system analysis [5]. ML PF methods use historical operation data gathered from the metering infrastructure and try to approximate the power system state based on them. In [6], the authors effectively transformed the nonlinear PF relationship into a linear mapping within a higher-dimensional state space, resulting in a substantial improvement in the precision of the calculation process. Similarly, Chen et al. [7] mitigate the errors of model-based PF linearization approaches by approximating the nonlinear PF equations in a data-driven manner. For instance, in [8], the data-driven PF method comprises two distinct stages: offline learning and online computing. During the offline learning stage, a learning model is developed utilizing the proposed exact linear regression equations, which is subsequently solved by applying the ridge regression method to mitigate the impact of data collinearity. Subsequently, the need for nonlinear iterative calculations is obviated in the online computing stage, thereby streamlining the computation process. Furthermore, Liu et al. [9] reformulated PF into a regression model, leveraging the structural attributes of AC power flow equations, including Jacobian matrix-guided constraints, to substantially reduce the search space. Overall, all these simple ML methods have outperformed the DCPF method [10]; however, the scalability issues still do not allow for the application of these methods in large power systems.

Numerous deep learning techniques have been developed, promising to find more scalable solutions. Many studies emphasize the use of physics-informed methodologies that leverage the inherent characteristics of the problem. For example, Hu et al. [11] incorporate an auxiliary task for PF model

This work used the Dutch national e-infrastructure with the support of the SURF Cooperative using grant no. EINF-6262. This publication is part of the project ALIGN4energy (with project number NWA.1389.20.251) of the research programme NWA ORC 2020 which is (partly) financed by the Dutch Research Council (NWO). Stavros is by the HORIZON Europe Drive2X Project 101056934.

Nan Lin, Stavros Orfanoudakis, and Pedro P. Vergara are with the Intelligent Electrical Power Grids (IEPG) Section, Delft University of Technology, Delft, The Netherlands (emails: n.lin, s.orfanoudakis, p.p.vergarabarrios@tudelft.nl).

Nathan Ordonez Cardenas is with the Computer Science Department, Delft University of Technology, Delft, The Netherlands (email: n.a.ordonezcardenas@student.tudelft.nl).

Juan S. Giraldo is with the Energy Transition Studies Group, Netherlands Organization for Applied Scientific Research (TNO), Amsterdam, The Netherlands (email: juan.giraldo@tno.nl)

The first two authors contributed equally.

PowerFlowNet：利用消息传递图神经网络改进潮流近似

IEEE 学生会员 Nan Lin、IEEE 学生会员 Stavros Orfanoudakis、IEEE 学生会员 Nathan Ordonez Cardenas
Juan S. Giraldo，IEEE 会员；Pedro P. Vergara，IEEE 高级会员

2023 年 11 月 6 日 [cs.LG] 2311.03415v1 :2311.03415v1 [cs.LG] 2023 年 11 月 6 日

摘要——准确高效的潮流（PF）分析对于现代电网的高效运行和规划至关重要。因此，需要能够处理大规模电网并提供准确快速解决方案的可扩展算法。图神经网络（GNN）因其能够从底层电网图中捕捉独特特征，已成为提高潮流近似速度的一种很有前景的方法。本研究提出了一种新型的 GNN 架构 PowerFlowNet，用于潮流近似。PowerFlowNet 在简单的 IEEE 14 节点系统中展现出与传统牛顿-拉夫逊法相近的性能，但速度提高了 4 倍；在法国高压电网（6470rte）的实际案例中，速度更是提高了 145 倍。同时，PowerFlowNet 在性能和执行时间方面也显著优于其他传统近似方法，例如直流松弛法；因此，PowerFlowNet 是一种极具前景的实际潮流分析解决方案。此外，我们通过深入的实验评估验证了我们方法的有效性，全面考察了 PowerFlowNet 的性能、可扩展性、可解释性和架构可靠性。该评估有助于深入了解图神经网络在电力系统分析中的行为和潜在应用。

索引词——潮流状态估计、深度学习、图神经网络。

我.

电力系统的复杂性不断增加，这主要归因于分散式可再生能源的大量并网。在此背景下，潮流计算（PF）成为确保电力系统稳定性的根本挑战，在电网的运行管理和长期规划中都发挥着关键作用。潮流计算本质上是一个数学问题，其核心是确定各母线的电压，这需要求解一组与电网配置密切相关的非线性方程组。负荷分布和发电特性[1]。传统的计算方法，例如牛顿-拉夫逊法[2]、高斯-赛德尔法[3]和快速解耦法，虽然具有优异的精度和收敛性，但对于规模更大的电力系统，尤其是在对拥有数千个节点的国家电网进行长期规划时，其计算速度会显著降低。相比之下，直流潮流计算（DCPF）技术[4]通过对电压幅值进行假设，将该问题简化为线性问题，但代价是精度降低。因此，迫切需要能够高效求解规模显著更大的网络潮流计算的创新算法。这些新型算法应力求在保证精度的前提下兼顾速度，最终促进大型实际电网的高效运行和规划。

通过引入精确的智能计量和数据采集系统实现电网现代化，推动了机器学习（ML）方法在电力系统分析中的应用，从而实现更精确、更高效的电力系统分析[5]。ML 功率因数预测（PF）方法利用从计量基础设施收集的历史运行数据，并尝试基于这些数据来近似电力系统状态。文献[6]有效地将非线性功率因数关系转化为高维状态空间中的线性映射，显著提高了计算精度。类似地，Chen 等人[7]通过数据驱动的方式近似非线性功率因数方程，从而降低了基于模型的功率因数线性化方法的误差。例如，文献[8]中的数据驱动功率因数预测方法包含两个不同的阶段：离线学习和在线计算。在离线学习阶段，利用所提出的精确线性回归方程构建学习模型，然后应用岭回归方法求解该模型，以降低数据共线性的影响。随后，在在线计算阶段，无需进行非线性迭代计算，从而简化了计算过程。此外，Liu 等人[9]将潮流计算重新表述为回归模型，利用交流潮流方程的结构特性（包括雅可比矩阵引导的约束），大幅缩小了搜索空间。总体而言，所有这些简单的机器学习方法都优于 DCPF 方法[10]；然而，可扩展性问题仍然限制了这些方法在大型电力系统中的应用。

众多深度学习技术已被开发出来，有望找到更具可扩展性的解决方案。许多研究强调使用基于物理原理的方法，以利用问题的固有特征。例如，Hu 等人 [11] 在 PF 模型中引入了一个辅助任务。

本研究利用了荷兰国家电子基础设施，并得到了 SURF 合作组织（项目编号 EINF-6262）的支持。本出版物是 ALIGN4energy 项目（项目编号 NWA.1389.20.251）的一部分，该项目隶属于 NWA ORC 2020 研究计划，该计划由荷兰研究理事会（NWO）部分资助。Stavros 的研究工作得到了 HORIZON Europe Drive2X 项目（项目编号 101056934）的支持。

Nan Lin、Stavros Orfanoudakis 和 Pedro P. Vergara 均来自荷兰代尔夫特理工大学智能电网 (IEPG) 部门（电子邮件：n.lin、s.orfanoudakis、ppvergarabarrios@tudelft.nl）。

Nathan Ordonez Cardenas 就职于荷兰代尔夫特理工大学计算机科学系（邮箱：naordonezcardenas@student.tudelft.nl）。

Juan S. Giraldo 就职于荷兰应用科学研究组织 (TNO) 能源转型研究组，地址：荷兰阿姆斯特丹（邮箱：juan.giraldo@tno.nl）。前两位作者贡献相同。

reconstruction, wherein the neural network (NN) based PF solver is effectively regularized by encoding varying levels of Kirchhoff's laws and system topology into the reconstructed PF model, consequently ensuring adherence to physical laws and constraints. In [12], the training process of the NN is enhanced through a training process guided by physical PF equations, such as incorporating branch flows as a penalty term in the NN's objective function and simplifying the backpropagation update gradients based on transmission grid's physical characteristics to expedite training without compromising effective model guidance. Similarly, Li et al. [13] use NNs to estimate the distribution system state equations by creating virtual nodes to represent nodes without any smart metering. An alternative approach is to enhance a classical solver (Gauss-Newton) by training an NN to serve as a regularizer [14]. However, it is important to note that classic deep learning approaches lack permutation equivariance, leading to limited generalization capabilities and reduced efficiency in real-world scenarios.

GNNs [15] have garnered significant attention in recent years due to their capacity to leverage graph-structured data by aggregating information from neighboring nodes while also being permutation equivariant, i.e., allowing the same NN to be used for problems with a different number of nodes. Consequently, researchers have explored diverse GNN architectures to tackle the PF problem, given that the grid topology often exhibits a sparsity pattern resembling a graph with useful connectivity information. In particular, GNNs have been successful in various tasks related to power systems [16], such as fault scenario application, time-series prediction, power flow calculation, and data generation. Wu et al. [17] utilized graph convolution networks (GCN) and recurrent NNs inspired by the physics of power systems to derive the PF equations. In [18], a weakly supervised learning approach is employed based on the PF equations that do not require labeled data, however, at the cost of accuracy compared to fully supervised methods. Furthermore, Pagnier et al. [19] improved some improvements on the previous method, such as implementing a specific encoder-decoder loss for the prediction of effective PF model parameters. Meanwhile, [20] improves upon an iterative-based GNN approach by replacing the pre-computed loss values with a purely physics-constraint loss. Despite the proliferation of various studies in this domain, little attention has been directed toward investigating the effectiveness of proposed methodologies for large power networks. Consequently, a comprehensive evaluation and comparison of ML and GNN-based approaches concerning accuracy, computational complexity, scalability, and interpretability are imperative to discern the most promising techniques for PF analysis.

To address the scalability and other aforementioned real-world application concerns, we propose PowerFlowNet. PowerFlowNet is a novel GNN architecture specifically designed to leverage electrical power networks' structural characteristics and interconnectedness, enabling efficient approximation of the PF. In practice, PowerFlowNet transforms the PF into a GNN node-regression problem by representing each bus as a node and each transmission line as an edge while maintaining the network's connectivity. The key advantage of PowerFlowNet lies in its ability to swiftly and accurately predict the complete PF, regardless of the network size; thus, enabling it to be applied for realistic power grid planning and operation processes. Specifically, our thorough experimental evaluation demonstrates PowerFlowNet's accuracy and execution speed compared to traditional methods, such as DCPF, while its performance is similar to the most trusted Newton-Raphson method but with significantly lower execution time. The much lower execution time will be extremely useful in particular scenarios where it is required to estimate thousands of PF in a short duration, given the high uncertainty introduced by distributed energy resources. Additionally, an in-depth analysis of the proposed approach's scalability, architectural dependability, and interpretability is conducted, aiming to provide valuable insights into the broader applicability of GNN algorithms in the power system domain.

In detail, the key contribution of this paper is summarized as follows:

- We introduce a novel GNN architecture for high-quality PF approximation with up to two orders of magnitude speedup compared to the Newton-Raphson method. Its distinctiveness, compared to existing PF GNN approaches, lies in its adept utilization of the capabilities from message passing GNNs [21] and high-order GCN [22], in a unique arrangement called PowerFlowConv, for handling a trainable masked embedding of the network graph. This innovative approach renders the proposed GNN architecture remarkably scalable, presenting an effective solution for the PF problem.

II. GNNs and Classic PF Algorithms

This section provides an overview of the fundamental background of GNNs and the methodologies they employ. Furthermore, it delves into the traditional techniques used in PF analysis, including the Newton-Raphson and DCPF methods.

A. Graph Neural Networks

GNNs are a class of ML models designed to operate on graph-structured data, which can represent complex relationships and interactions between entities. GNNs have gained significant attention due to their ability to capture and leverage the structural information inherent in graphs. In detail, a graph $\mathcal{G}(\mathcal{N}, \mathcal{E})$ can be defined as a set of $\mathcal{N}$ nodes and $\mathcal{E}$ edges with node features $\boldsymbol{x}_i \in \mathbb{R}^F$ and edge features $\boldsymbol{x}^e_{i,j} \in \mathbb{R}^{F_e}$, where $i \in \mathcal{N}$ and the pair $(i, j) \in \mathcal{E}$. More specifically, GNNs use parts of regular NNs to help them understand and work with the structure of these graphs.

1) Message Passing NNs: At the heart of GNNs lies the concept of message passing [21]. Message passing allows information to propagate through the graph by iteratively aggregating and updating node features based on their neighborhood relationships. In each message passing step, each node receives information from its neighbors, performs local computations, and then transmits updated information to its neighbors (Fig. 1). Specifically, the message passing phase runs for T timesteps for every node with hidden state $\boldsymbol{h}^t_i$

Hu 等人提出的重构方法，通过将不同层次的基尔霍夫定律和系统拓扑结构编码到重构的 PF 模型中，有效地对基于神经网络（NN）的 PF 求解器进行正则化，从而确保其符合物理定律和约束条件。在文献[12]中，通过物理 PF 方程指导的训练过程增强了 NN 的训练效果，例如将支路流量作为惩罚项纳入 NN 的目标函数，并基于输电网的物理特性简化反向传播更新梯度，以加快训练速度，同时又不影响模型的有效指导。类似地，Li 等人[13]使用 NN 通过创建虚拟节点来表示没有智能计量的节点，从而估计配电系统状态方程。另一种方法是通过训练 NN 作为正则化器来增强经典求解器（高斯-牛顿法）[14]。然而，需要注意的是，经典的深度学习方法缺乏置换等变性，导致其泛化能力有限，并且在实际场景中的效率较低。

近年来，图神经网络（GNN）[15] 因其能够利用图结构数据聚合相邻节点的信息，同时又具有置换等变性（即允许同一个神经网络用于节点数量不同的问题）而备受关注。鉴于电网拓扑结构通常呈现出类似于图的稀疏模式，并包含有用的连接信息，研究人员探索了各种 GNN 架构来解决潮流计算（PF）问题。特别是，GNN 已成功应用于与电力系统相关的各种任务[16]，例如故障场景应用、时间序列预测、潮流计算和数据生成。Wu 等人[17] 利用受电力系统物理启发的图卷积网络（GCN）和循环神经网络推导了潮流计算方程。文献[18] 采用了一种基于潮流计算方程的弱监督学习方法，该方法不需要标记数据，但与全监督方法相比，精度有所降低。此外，Pagnier 等人[19]对先前的方法进行了一些改进，例如实现了特定的编码器-解码器损失函数来预测有效的 PF 模型参数。同时，[20]通过用纯粹的物理约束损失函数替换预先计算的损失值，改进了基于迭代的 GNN 方法。尽管该领域的研究层出不穷，但很少有研究关注已提出的方法在大电网中的有效性。因此，对基于机器学习和 GNN 的方法进行全面的评估和比较，包括准确性、计算复杂度、可扩展性和可解释性，对于甄别最有前景的 PF 分析技术至关重要。

为了解决可扩展性及其他上述实际应用问题，我们提出了 PowerFlowNet。PowerFlowNet 是一种新型的图神经网络 (GNN) 架构，旨在充分利用电力网络的结构特性和互联性，从而高效地逼近潮流 (PF)。在实际应用中，PowerFlowNet 将潮流问题转化为 GNN 节点回归问题，将每个母线表示为一个节点，每条输电线路表示为一条边，同时保持网络的连通性。PowerFlowNet 的主要优势在于其能够快速准确地预测完整的功率因数，且不受网络规模的限制；因此，它能够应用于实际的电网规划和运行过程中。具体而言，我们全面的实验评估表明，与 DCPF 等传统方法相比，PowerFlowNet 具有更高的精度和执行速度；其性能与最可靠的牛顿-拉夫逊法相当，但执行时间却显著降低。考虑到分布式能源带来的高度不确定性，在需要在短时间内估计数千个功率因数的特定场景下，显著降低的执行时间将非常有用。此外，我们还对所提出方法的可扩展性、架构可靠性和可解释性进行了深入分析，旨在为 GNN 算法在电力系统领域的更广泛应用提供有价值的见解。

具体而言，本文的主要贡献总结如下：

- 我们提出了一种新型的图神经网络（GNN）架构，用于高质量地逼近功率流（PF）问题，其速度比牛顿-拉夫逊法提高了两个数量级。与现有的功率流 GNN 方法相比，该架构的独特之处在于它巧妙地利用了消息传递 GNN[21]和高阶图卷积网络（GCN）[22]的能力，并以一种名为 PowerFlowConv 的独特方式来处理网络图的可训练掩码嵌入。这种创新方法使得所提出的 GNN 架构具有显著的可扩展性，为功率流问题提供了一种有效的解决方案。

二、图神经网络特征预测与分析 (GNNCPF)

本节概述了图神经网络的基本原理及其所采用的方法。此外，还深入探讨了特征预测分析中使用的传统技术，包括牛顿-拉夫逊法和动态特征预测与分析 (DCPF) 方法。

A. 图神经网络

图神经网络（GNN）是一类机器学习模型，旨在处理图结构数据，这些数据能够表示实体之间复杂的关联和交互。GNN 因其能够捕获和利用图中固有的结构信息而备受关注。具体来说，图 G(N, E) 可以定义为包含 N 个节点和 E 条边的集合，其中节点特征 $x \in R$，边特征 $x \in R$，$i \in N$，且 $(i, j) \in E$。更具体地说，GNN 利用常规神经网络的部分特性来帮助理解和处理这些图的结构。

1) 消息传递神经网络：图神经网络的核心在于消息传递的概念[21]。消息传递允许信息通过迭代地聚合和更新基于邻域关系的节点特征，在图中传播。在每个消息传递步骤中，每个节点从其邻居接收信息，执行本地计算，然后将更新后的信息传递给其邻居（图 1）。具体来说，对于每个具有隐藏状态 h 的节点，消息传递阶段运行 T 个时间步。

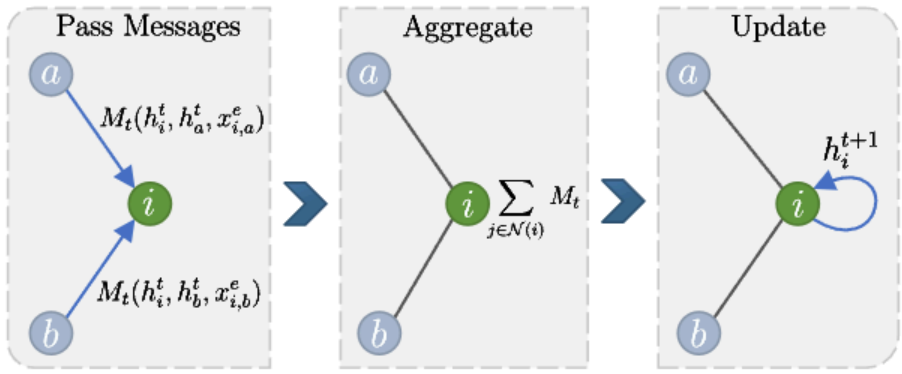


Fig. 1. A message passing step of node i consisting of message creation, aggregation, and update of the hidden state.

that is updated based on the message function $M_t(\cdot)$ and the node update function $U_t(\cdot)$. Therefore, for each node i with neighbors $\mathcal{N}(i)$, each iterative message update $\boldsymbol{v}_i^{t+1}$ is

$$\boldsymbol{v}_i^{t+1} = \sum_{j\in\mathcal{N}(i)} M_t(\boldsymbol{h}_i^t, \boldsymbol{h}_j^t, \boldsymbol{x}_{i,j}^e), \tag{1}$$

$$\boldsymbol{h}_i^{t+1} = U_t(\boldsymbol{h}_i^t, \boldsymbol{v}_i^{t+1}). \tag{2}$$

As shown in (1), the edge features $\boldsymbol{x}_{i,j}^e$ can also be included in the message creation step. After the message passing phase ends, the final feature vector for the whole graph is denoted as the output of a function $R(\cdot)$

$$\hat{\boldsymbol{y}}(\mathcal{G}) = R(\{\boldsymbol{h}_i^{\mathsf{T}} | i \in \mathcal{G}\}). \tag{3}$$

Notice that the functions $M_t(\cdot), U_t(\cdot)$, and $R(\cdot)$ are designed by the user and should be fully differentiable so that they can be trained and used by graph ML tasks. In addition, $R(\cdot)$ is often designed to be invariant to permutations of the set of node states to satisfy the equivariant feature of GNNs. Overall, message passing enables all nodes to gather and integrate information from their local context, incorporating both the features of neighboring nodes and the graph structure.

2) Graph Convolutional Networks: One other popular type of GNN is the GCNs [15], which introduces convolution operations on graphs. Similar to convolution neural networks (CNNs) for regular grid-like data, GCNs apply filters to node features and aggregate information from neighboring nodes. By leveraging the local connectivity of the graph, GCNs can capture both node-level patterns and the information of the K-hop neighbors. In particular, a graph convolutional layer of order K is defined as

$$\hat{\boldsymbol{y}}(S, X) = \sigma\left(\sum_{k=0}^{K-1} S^k X W\right). \tag{4}$$

Here, a GCN layer is a function of the graph shift operator $S \in \mathbb{R}^{N\times N}$, node features $X \in \mathbb{R}^{N\times F}$, and trainable weight matrix $W \in \mathbb{R}^{F\times F}$. Matrix S can be the adjacency matrix A, or a more complex representation such as the Laplacian transformation L. Moreover, σ represents a non-linear function, such as rectified linear units (ReLU), which is employed to enable the approximation of every target non-linear function. The parameter K holds significance as it dictates the K-hop[1] neighboring nodes are taken into account during each node convolution. Consequently, a larger value of K entails the inclusion of more nodes.

[1]Nodes with distance K from the original node.

3) Topology Adaptive GCN: Many variants of the GCN layer have been proposed with one of them being the topology adaptive graph convolution operator (TAGConv) [22]. TAGConv addresses a limitation of traditional GCN by adaptively learning the importance of different neighboring nodes during message passing. TAGConv assigns weights to the neighbors based on their relevance to the target node. Specifically, a TAGConv layer of order K is defined as

$$\hat{\boldsymbol{y}}(S, X) = \sigma\left(\sum_{k=0}^{K-1} S^k X W_k\right). \tag{5}$$

Here, the trainable weight matrix $W_k \in \mathbb{R}^{F\times F}$ is different for every K. This adaptivity enables TAGConv to effectively capture the local structure and important dependencies in the graph, resulting in improved performance in various tasks.

B. Classic Power Flow Analysis Methods

The PF problem is mathematically formulated as solving a set of non-linear equations. A power system has a set of buses (nodes) $\mathcal{N}$ and a set of transmission lines (undirected edges) $\mathcal{E} \subseteq \mathcal{N} \times \mathcal{N}$. For each node $i \in \mathcal{N}$, the PF equations are derived from Kirchhoff's current law and are expressed in complex form as

$$P_i^{\mathrm{L}} - P^{\mathrm{G}} + j(Q_i^{\mathrm{L}} - Q^{\mathrm{G}}) = \sum_{j\in\mathcal{N}(i)} \dot{V}_i \left(\frac{\dot{V}_j - \dot{V}_i}{r_{i,j} + jx_{i,j}}\right)^* \tag{6}$$

where P_i and Q_i are the active and reactive power drawn from bus i to the ground, $\dot{V}_i = V_i^{\mathrm{m}}\angle\theta_i$ and $\dot{V}_j = V_j^{\mathrm{m}}\angle\theta_j$ are the voltage phasors at bus i and j, and $z_{ij} = r_{ij} + x_{ij}$ are the line impedance associated with edge $(i, j) \in \mathcal{E}$. At each node i, two of P_i, Q_i, V_i^{m}, θ_i are unknown. For a system with $|\mathcal{N}| = N$ nodes, there are $2N$ real-valued equations and $2N$ unknown real-valued variables. For simplicity, we denote $M = 2N$. The vector form is represented as

$$\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{0} \tag{7}$$

where $\boldsymbol{x} \in \mathbb{R}^M$ containing every unknown variable at all nodes and $\boldsymbol{f} : \mathbb{R}^M \to \mathbb{R}^M$.

1) Newton-Raphson Algorithm: The Newton-Raphson algorithm [2] is an iterative method for solving non-linear equations. It starts with an initial guess $\boldsymbol{x}^{(0)} = (x_1^{(0)}, x_2^{(0)}, ..., x_M^{(0)})$ and at every step, it updates the solution with a correction term

$$\boldsymbol{x}^{(t)} = \boldsymbol{x}^{(t-1)} + \Delta\boldsymbol{x}^{(t)}. \tag{8}$$

It calculates the update given $\boldsymbol{x}^{(t-1)}$ and a correction, assuming

$$\boldsymbol{f}(\boldsymbol{x}^{(t-1)} + \Delta\boldsymbol{x}^{(t)}) = \boldsymbol{0}. \tag{9}$$

With one order of Taylor approximation, we can get the following linear equations.

$$\boldsymbol{f}(\boldsymbol{x}^{(t-1)}) = -\mathbf{J}(\boldsymbol{f}(\boldsymbol{x}^{(\boldsymbol{t-1})})) \cdot \Delta\boldsymbol{x}^{(t)}. \tag{10}$$

Here, $\mathbf{J}(\boldsymbol{f}(\boldsymbol{x}^{(t-1)}))$ is the Jacobian matrix of $\boldsymbol{f}$ over $\boldsymbol{x}$ evaluated at $\boldsymbol{x}^{(t-1)}$. In every step, we need to solve the above equations once, involving an $M \times M$ matrix inversion. Newton-Raphson method usually converges very fast in the power flow problem, but because of the inversion, the computation time grows quickly with the number of nodes [23].

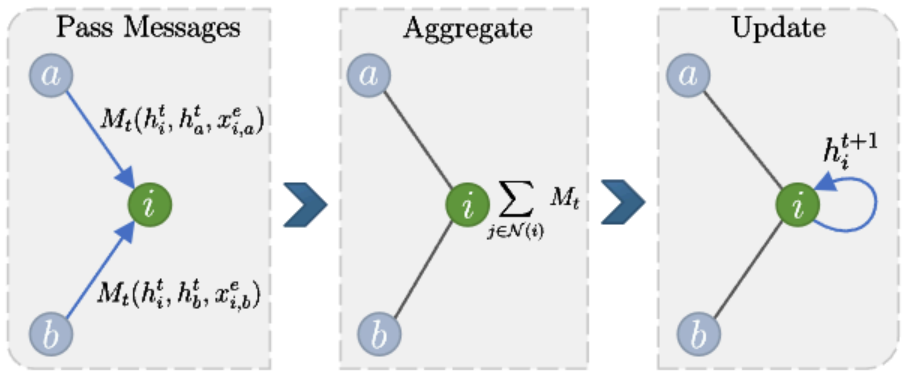


图 1. 节点 i 的消息传递步骤，包括消息创建、聚合和隐藏状态更新。

该更新基于消息函数 M(・) 和节点更新函数 U(・)。因此，对于每个具有邻居 N(i) 的节点 i，每次迭代消息更新 vi 为

$$\text{我们} = \sum_{j \in N(i)} M(h, h, x), \tag{1}$$

$$hi = U(h, vi)。 \tag{2}$$

如式 (1) 所示，边特征 x 也可以包含在消息创建步骤中。消息传递阶段结束后，整个图的最终特征向量表示为函数 R(・) 的输出。

$$\hat{y}(G) = R(\{h | i \in G\})。 \tag{3}$$

请注意，函数 M(・)、U(・) 和 R(・) 由用户设计，并且应完全可微，以便能够训练并用于图机器学习任务。此外，R(・) 通常被设计为对节点状态集的排列保持不变，以满足图神经网络的等变特性。总而言之，消息传递使所有节点能够收集并整合来自其局部上下文的信息，从而融合相邻节点的特征和图结构。

2) 图卷积网络：另一种流行的类型

图神经网络（GCN）[15] 是图神经网络的一种，它引入了图上的卷积运算。与用于规则网格状数据的卷积神经网络（CNN）类似，GCN 对节点特征应用滤波器，并聚合来自相邻节点的信息。通过利用图的局部连通性，GCN 可以同时捕获节点级模式和 K 跳邻居的信息。具体而言，K 阶图卷积层定义为：

$$\hat{y}(S, X) = \sigma\left(\sum_{k=0}^{K-1} SXW\right). \tag{4}$$

这里，GCN 层是图移位算子 S ∈ R、节点特征 X ∈ R 和可训练权重矩阵 W ∈ R 的函数。矩阵 S 可以是邻接矩阵 A，也可以是更复杂的表示，例如拉普拉斯变换 L。此外，σ表示一个非线性函数，例如修正线性单元（ReLU），用于逼近每个目标非线性函数。参数 K 至关重要，因为它决定了每次节点卷积时考虑的 K 跳邻域节点数。因此，K 值越大，包含的节点就越多。

[1] 与原始节点距离为 K 的节点。

3) 拓扑自适应 GCN：GCN 的多种变体

已经提出了多种层，其中之一是拓扑自适应图卷积算子（TAGConv）[22]。TAGConv 通过在消息传递过程中自适应地学习不同邻居节点的重要性，解决了传统图卷积网络（GCN）的局限性。TAGConv 根据邻居节点与目标节点的相关性为其分配权重。具体来说，阶数为 K 的 TAGConv 层定义为：

$$\hat{y}(S, X) = \sigma\left(\sum_{k=0}^{K-1} SXW\right). \tag{5}$$

这里，可训练权重矩阵 W∈Ris 对于每个 K 都不同。这种自适应性使得 TAGConv 能够有效地捕获图中的局部结构和重要依赖关系，从而在各种任务中提高性能。

B. 经典潮流分析方法

功率流问题在数学上被表述为求解一组非线性方程。一个电力系统包含一组母线（节点）N 和一组输电线路（无向边）E ⊆ N × N。对于每个节点 i ∈ N，功率流方程由基尔霍夫电流定律推导而来，并以复数形式表示为：

$$P - P + j(Q - Q) = \sum_{j \in N(i)} V \overline{\frac{V - \cdot V}{r + jx}} \tag{6}$$

其中 P 和 Q 分别表示从母线 i 到地的有功功率和无功功率，V = V∠θ 和 V = V∠θ 分别表示母线 i 和 j 处的电压相量，z = r + x 表示边 (i, j) ∈ E 的线路阻抗。在每个节点 i，P、Q、V、θ 中有两个未知。对于一个具有 |N| = N 个节点的系统，存在 2N 个实值方程和 2N 个未知实值变量。为简便起见，我们记 M = 2N。向量形式表示为：

$$f(x) = 0 \tag{7}$$

其中 x ∈ R 包含所有节点上的所有未知变量，并且 f : R→ R。

1) 牛顿-拉夫逊算法：牛顿-拉夫逊算法

算法[2]是一种求解非线性方程的迭代方法。它从初始猜测值 x= (x1, x2, ..., xM)开始，并在每一步中用修正项更新解。

$$x = x + \Delta x。 \tag{8}$$

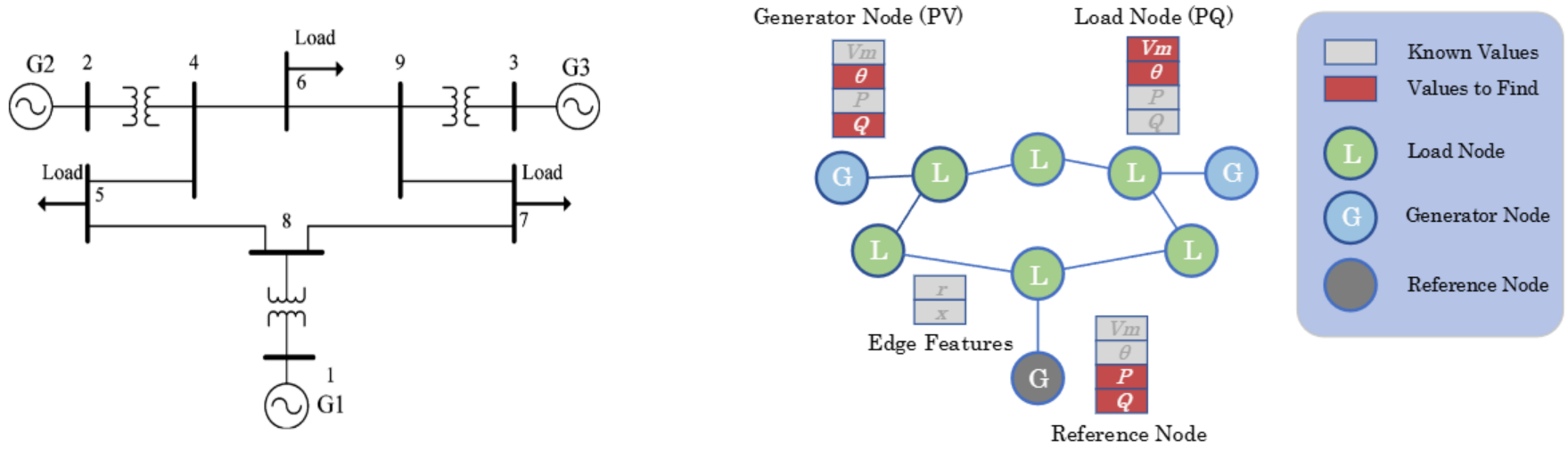
它根据 x 和修正值计算更新，假设

$$f(x + \Delta x) = 0。 \tag{9}$$

利用一阶泰勒近似，我们可以得到以下线性方程。

$$f(x) = -J(f(x)) \cdot \Delta x。 \tag{10}$$

这里，J(f(x)) 是函数 f 在 x 处的雅可比矩阵。每一步都需要求解上述方程一次，涉及 M × M 矩阵求逆。牛顿-拉夫逊法在潮流计算问题中通常收敛速度很快，但由于需要进行矩阵求逆运算，计算时间会随着节点数的增加而迅速增长 [23]。



(a) Standard IEEE 9-case. (b) GNN equivalent of the IEEE 9-case.

Fig. 2. Interpreting the PF problem of the IEEE 9-case into a GNN node regression problem. In detail, Fig. 2a shows the standard IEEE 9-case, and Fig. 2b illustrates how each bus is transformed into a load, generator, or reference node while keeping the same line connectivity. Each node has different known and unknown features $\boldsymbol{x} = (V^m, \theta, P, Q)$ depending on its type. The ultimate goal of this GNN problem is to approximate all the features for all nodes, thus solving the PF problem.

2) DC Power Flow Approximation: DCPF is an approximation of the ACPF. In DCPF, the voltage is assumed constant at all buses; therefore, the voltage and reactive power differences are 0, so the ΔV and ΔQ terms are neglected [4]. Subsequently, the PF equations can be further simplified to a linear problem that does not require an iterative solution. In detail, the relation between the active power (P_i) and the voltage angle (θ_i) of each bus is

$$P_i = \Re\left[\sum_{j\in\mathcal{N}(i)} \dot{V}_i \left(\frac{\dot{V}_j - \dot{V}_i}{r_{i,j} + jx_{i,j}}\right)^*\right] \approx \sum_{j\in\mathcal{N}(i)} \frac{\theta_j - \theta_i}{x_{i,j}}. \tag{11}$$

III. PowerFlowNet Framework

This section presents a thorough explanation of PowerFlowNet, accompanied by our interpretation of PF formulation in the domain of GNN algorithms.

A. Interpreting Power Flow as a GNN Problem

The primary objective of a PF study is to determine the voltage magnitudes and angles for a given load, generation, and network state, facilitating subsequent calculations of line flows and losses. This problem lends itself seamlessly to a GNN approach, as the electrical network topology can be represented as an undirected graph $\mathcal{G}(\mathcal{N}, \mathcal{E})$, where the set of nodes $\mathcal{N}$ corresponds to all the buses and the set of edges $\mathcal{E}$ represents power lines. The node features, denoted as $X \in \mathbb{R}^{N\times F}$, encompass various attributes for each bus, including voltage magnitude (V_i^{m}), voltage angle (θ_i), active power (P_i), and reactive power (Q_i). Similarly, the edge features, denoted as $X^e \in \mathbb{R}^{E\times F_e}$, encompass the resistance (r_{ij}) and reactance (x_{ij}) attributes for each line $(i, j) \in \mathcal{E}$.

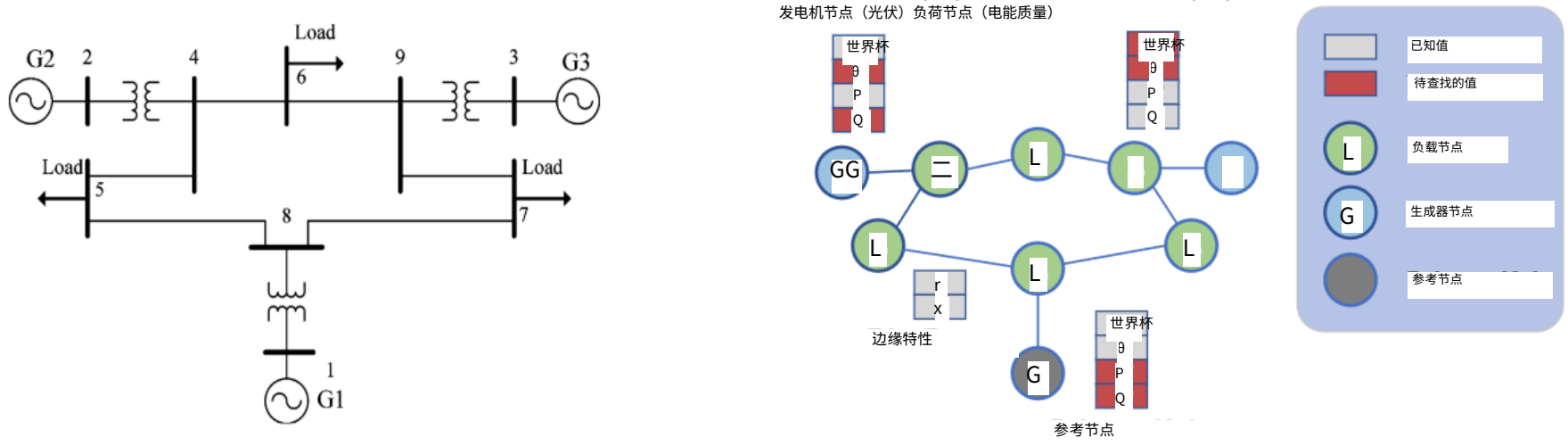
Illustrated in Fig. 2, the conversion of a transmission network into a graph representation involves mapping the buses to nodes while preserving the interconnectedness of the lines. The nodes are categorized into three types: a) load (PQ) nodes where the P_i and Q_i are known, b) generator (PV) nodes where V_i^{m} and P_i are known, and c) reference nodes where both V_i^{m} and θ_i are known. The edge features X^e are assumed to be given. The objective of this task is to predict the missing node features for every node $i \in \mathcal{N}$, thereby rendering this problem as an intriguing node regression task.

B. PowerFlowNet Architecture

PowerFlowNet is a GNN approach that reconstructs every node's full feature vector $\hat{\boldsymbol{x}}_i = (V_i^{\mathrm{m}}, \theta_i, P_i, Q_i)$ given partial information of the problem, such as the adjacency matrix of the graph A, the known node features $X = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]^\mathsf{T}$, and the edge features $X^e = [\boldsymbol{x}^e{}_{0,0}, \ldots, \boldsymbol{x}^e{}_{i,j}]^\mathsf{T}$. Our proposed model consists of a mask encoder and a stack of our novel Power Flow Convolutional layers (PowerFlowConv). Initially, the mask encoding layers transform the input features into a continuous embedding using a feature mask as a parameter. Then, as illustrated in Fig. 3, the encoded graph features are fed to the stack of Power Flow Convolutional operations consisting of a unique arrangement of message passing and TAGConv layers. This way, information from every node and edge is aggregated so that complete feature matrix $\hat{X} = [\hat{\boldsymbol{x}}_1, \ldots, \hat{\boldsymbol{x}}_N]^\mathsf{T}$ is predicted. A detailed explanation of PowerFlowNet's components and procedures is presented next.

1) Mask Encoder: In the PF problem, each node has different known and unknown features. The goal is to predict the unknown features while keeping the known ones unchanged. This means that our NN should know which features must be predicted. Consequently, for every input node with feature vector $\boldsymbol{x}_i$ we create a binary mask $\boldsymbol{m}_i \in \mathbb{R}^F$ where 0 represents the known and 1 the unknown features. For example, the mask of a load (PQ) node with a feature vector $\boldsymbol{x}_i = (V_i^m, \theta_i, P_i, Q_i)$ and unknown values V^m and θ, will be $\boldsymbol{m}_i = (1, 1, 0, 0)$. Additionally, similar to the positional embeddings in Transformers [24], we propose using a mask encoder, which in practice is an MLP with one hidden layer that learns a continuous embedding for every binary mask. The exact mathematical operation for every $m_i \in M$, where $M \in \mathbb{R}^{N\times F}$, is

$$\hat{\boldsymbol{m}}_i = W_1\sigma(W_0\boldsymbol{m}_i + b_0) + b_1, \ \forall i \in \mathcal{N}, \tag{12}$$



（a）标准 IEEE 9 型外壳。　　（b）IEEE 9 情况的 GNN 等效方案。

图 2. 将 IEEE 9 路并联系统的功率因数问题转化为 GNN 节点回归问题。具体而言，图 2a 展示了标准的 IEEE 9 路并联系统，图 2b 则展示了如何在保持线路连接不变的情况下，将每个母线转换为负载节点、发电机节点或参考节点。每个节点根据其类型具有不同的已知和未知特征 x = (V, θ, P, Q)。该 GNN 问题的最终目标是逼近所有节点的全部特征，从而求解功率因数问题。

2) 直流功率流近似：直流功率流近似是一种近似方法。ACPF 的实现。在 DCPF 中，假设所有母线的电压恒定；因此，电压差和无功功率差均为 0，所以忽略 ΔV 和 ΔQ 项 [4]。随后，PF 方程可以进一步简化为一个无需迭代求解的线性问题。具体而言，各母线的有功功率 (P) 与电压角 (θ) 之间的关系为：

$$P = \Re\left(\sum_{j \in N(i)} V \left(\frac{V - \cdot V}{r + jx} \right)^{!} \right) \approx \sum_{j \in N(i)} \frac{\theta - \theta}{x} \cdot \quad (11)$$

III. PFNF 本节对 PowerFlowNet 进行了详尽的解释，并结合 GNN 算法领域对 PF 公式的解释。

A. 将潮流问题解释为 GNN 问题

功率因数分析的主要目标是确定给定负荷、发电量和网络状态下的电压幅值和电压角，以便后续计算线路潮流和损耗。该问题非常适合采用图神经网络（GNN）方法，因为电网拓扑结构可以表示为一个无向图 G(N, E)，其中节点集 N 对应于所有母线，边集 E 对应于输电线路。节点特征（记为 X ∈ R）包含每个母线的各种属性，包括电压幅值 (V)、电压角 (θ)、有功功率 (P) 和无功功率 (Q)。类似地，边特征（记为 X ∈ R）包含每条线路 (i, j) ∈ E 的电阻 (r) 和电抗 (x) 属性。

如图 2 所示，将输电网络转换为图表示的过程涉及将母线映射到节点，同时保持线路之间的互连性。节点分为三类：a) 负荷 (PQ) 节点，其中 P 和 Q 已知；b) 发电机 (PV) 节点，其中 V 和 Pa 已知；c) 参考节点，其中 V 和 θ 均已知。

假设边特征 X 已知。此任务的目标是预测每个节点 i ∈ N 的缺失节点特征，因此该问题可以转化为一个有趣的节点回归任务。

B. PowerFlowNet 架构

PowerFlowNet 是一种图神经网络 (GNN) 方法，它利用问题的部分信息（例如图 A 的邻接矩阵、已知的节点特征 X = [x, ..., x] 和边特征 X = [x, ..., x]）重构每个节点的完整特征向量 ˆx = (V, θ, P, Q)。我们提出的模型由一个掩码编码器和一系列我们提出的新型电力流卷积层 (PowerFlowConv) 组成。首先，掩码编码层使用特征掩码作为参数，将输入特征转换为连续嵌入。然后，如图 3 所示，编码后的图特征被送入由消息传递层和 TAGConv 层独特排列的电力流卷积层堆栈。这样，来自每个节点和边的信息被聚合，从而得到完整的特征矩阵。

预测 $\hat{X} = [\hat{x}, \ldots, \hat{x}]$。接下来将详细解释 PowerFlowNet 的组件和流程。

1) 掩码编码器：在 PF 问题中，每个节点都有不同的已知特征和未知特征。目标是在保持已知特征不变的情况下预测未知特征。这意味着我们的神经网络需要知道哪些特征需要预测。因此，对于每个具有特征向量 x 的输入节点，我们创建一个二值掩码 m∈R，其中 0 表示已知特征，1 表示未知特征。例如，对于具有特征向量 x=(V, θ, P, Q)和未知值 V 和θ的负载(PQ)节点，其掩码为 m=(1, 1, 0, 0)。此外，类似于 Transformer 中的位置嵌入 [24]，我们提出使用掩码编码器，它实际上是一个具有一个隐藏层的多层感知器(MLP)，可以学习每个二值掩码的连续嵌入。对于每个 m∈M，其中 M∈R，其精确的数学运算如下：

$$\hat{m} = W\sigma(Wm + b) + b, \forall i \in N, \quad (12)$$

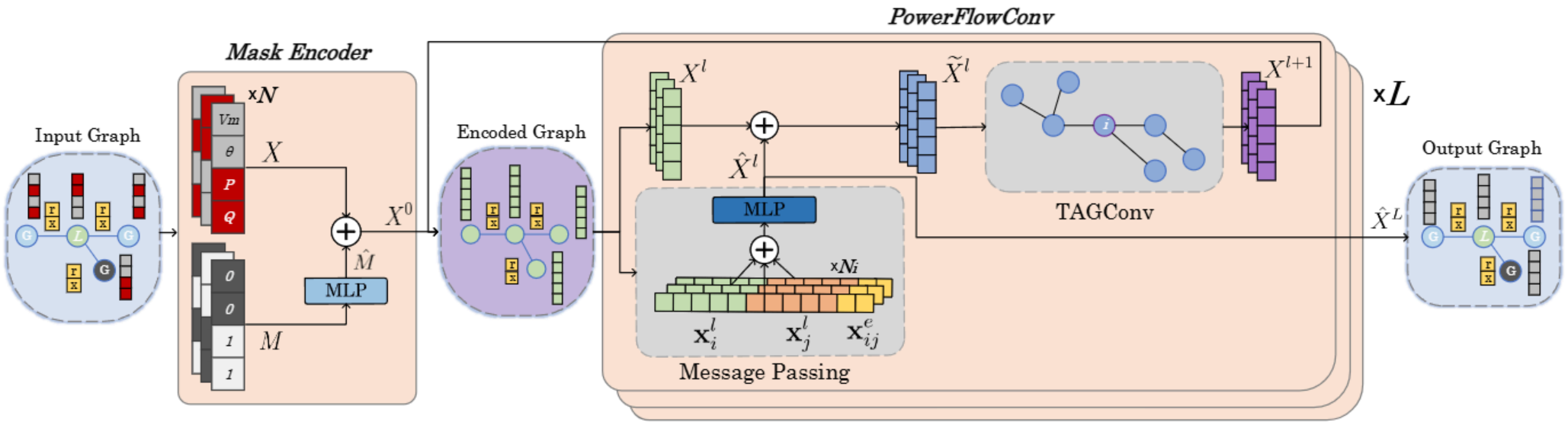


Fig. 3. The PowerFlowNet model architecture consists of a mask encoder and L PowerFlowConv layers. The input graph with incomplete feature information is fed node-by-node to the mask encoder to generate encoded graph features, where each node $n \in \mathcal{N}$ with $(\boldsymbol{x}_i, \boldsymbol{m}_i)$. Then, the encoded graph features are processed by a series of L sequential PowerFlowConv layers, each comprising a 1-step message passing and a high-order TAGConv. Finally, the complete output graph is produced.

where the function $\{0,1\}^F \rightarrow \mathbb{R}^F$, and the weight matrices W_0, W_1 and the biases b_0, b_1 are the basic trainable components of the Mask Encoder's MLP. Finally, to produce the encoded graph features X^l, the input node features X, and node mask embeddings are added up as

$$\boldsymbol{x}_i^0 = \boldsymbol{x}_i + \hat{\boldsymbol{m}}_i, \ \forall i \in \mathcal{N}. \tag{13}$$

2) Power Flow Convolutional Layer: The second part of our proposed architecture consists of L connected PowerFlowConv layers that sequentially process the encoded graph features and predict the final feature matrix $\hat{X}^L$, where $\hat{X}^L = [\hat{\boldsymbol{x}}_1^L, \ldots, \hat{\boldsymbol{x}}_N^L]^\mathsf{T}$ and $\hat{\boldsymbol{x}}_i^L = (V_i^\mathrm{m}, \theta_i, P_i, Q_i)$, $\forall i \in \mathcal{N}$. Each PowerFlowConv layer consists of an initial one-hop message passing step similar to [21] for extracting graph information, including the edge features and a K-hop TAGConv [22] on a higher-dimensional feature space. With the message passing step, we can effectively update our encoded graph features with information from the edge features, i.e., the line resistance and reactance, so that the TAGConv can efficiently exchange information in the K-hop neighborhood in one go while considering the edge features. Generally, graph convolutional layers, such as TAGConv do not use information from the edge features. Nevertheless, in the PF formulation, line characteristics play an important role. Therefore, by using message passing, we distill edge feature information into the node features that can later be used by TAGConv to provide high-quality PF approximations.

More specifically, the message passing equations $\forall i \in \mathcal{N}$ are described as

$$\hat{\boldsymbol{x}}_i^l = W_{MP_1}^l \cdot \sigma \left(W_{MP_0}^l \sum_{j \in \mathcal{N}_i} \langle \boldsymbol{x}_i^l, \boldsymbol{x}_j^l, \boldsymbol{x}_{i,j}^e \rangle + b_{MP_0}^l \right) + b_{MP_1}^l \tag{14}$$

where $\langle x_i^l, x_j^l, x_{i,j}^e \rangle$ is a concatenated vector of the described vectors fed to a two-layer MLP with an in-between ReLU activation function. Here again, the weight matrices and bias vectors $W_{MP_1}^l, b_{MP_1}^l, W_{MP_0}^l, b_{MP_0}^l$ are the trainable parameters of the MLP of layer l.

Afterward, the intermediate graph signals $\widetilde{X}^l$ of layer l are calculated as the sum of the encoded graph features X^l, and the processed message passing graph features $\hat{X}^l$, i.e. $\widetilde{X}^l = X^l + \hat{X}^l$, $\forall l \in \{0, \ldots, L\}$. Then, the signals $\widetilde{X}^l$ are processed by the TAGConv layer to generate the new encoded graph features X^{l+1} as in

$$X^{l+1} = \sigma \left(\sum_{k=0}^{K-1} (D^{-\frac{1}{2}} A D^{-\frac{1}{2}})^k \widetilde{X}^l W_k^l \right). \tag{15}$$

Here, $(D^{-1/2} A D^{-1/2})$ denotes the normalized adjacency matrix based on the diagonal degree matrix D. As shown in [22], the normalized adjacency matrix is often used in GNNs to provide computational stability guarantees. Then, the encoded graph features generated by the *l-th* PowerFlowConv layer are passed to the next layer until $l = L$. Finally, the message passing step is used in the last layer while the TAGConv step is discarded, as depicted in Fig. 3.

Notice that PowerFlowNet is equivariant to the permutation of the input nodes while also leveraging the edge features in every message passing step. Permutation equivariance refers to a property where the output of each node remains consistent regardless of the specific ordering of nodes in the input. This particular inductive bias arises naturally from the PF problem itself, as the PF equations (See Section II-B) also exhibit permutation equivariance. Additionally, we emphasize the importance of incorporating edge features into every step of our message passing process. This inclusion is crucial because edge features are fundamental variables within the PF equations, and their consideration is essential for making accurate predictions.

3) Loss Functions: The selection of the appropriate loss function is also important in generating high-quality PF approximations. Usually, purely physics-based losses are used in existing ML approaches for PF [18], [20]. However, using only physical losses is insufficient due to the peculiarities of the PF problem. Consequently, we conducted further experiments, which led us to utilize the Mean Squared Error (MSE) for the training of PowerFlowNet and develop the Masked L2 Loss as a better evaluation metric. In detail, MSE is defined as

$$\mathrm{MSE}\,(\boldsymbol{y}_i, \hat{\boldsymbol{x}}_i) = ||\boldsymbol{y}_i - \hat{\boldsymbol{x}}_i||_2^2 \tag{16}$$

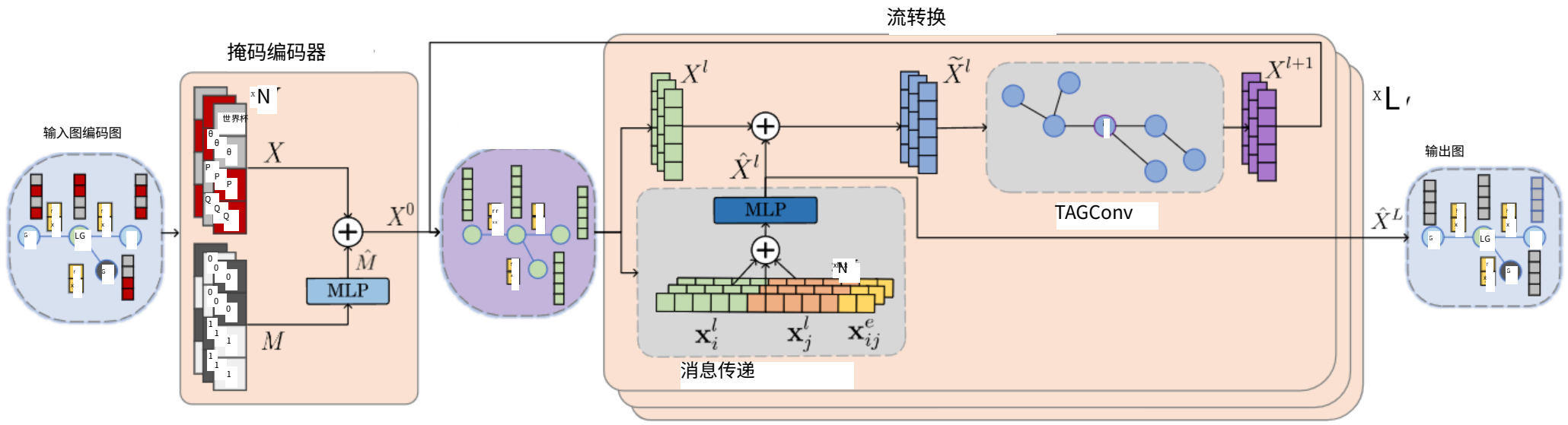


图 3. PowerFlowNet 模型架构由一个掩码编码器和 L 个 PowerFlowConv 层组成。输入图包含不完整的特征信息，并逐节点输入到掩码编码器中生成编码后的图特征，其中每个节点 n ∈ N 满足 (x, m)。然后，编码后的图特征由一系列 L 个顺序的 PowerFlowConv 层进行处理，每个层包含一个单步消息传递和一个高阶 TAGConv。最后，生成完整的输出图。

其中函数 {0, 1}→ R 以及权重矩阵 W、W 和偏置 b 构成了掩码编码器 MLP 的基本可训练组件。最后，为了生成编码后的图特征 X，将输入节点特征 X 和节点掩码嵌入相加。

$$x = x + \hat{m}, \forall i \in N 。 \quad (13)$$

2) 功率流卷积层：第二部分

我们提出的架构由 L 个连接的 PowerFlowConv 层组成，这些层按顺序处理编码后的图特征并预测最终的特征矩阵 ˆ X，其中

$\hat{X} = [\hat{x}, \ldots, \hat{x}]$且 $\hat{x} = (V, \theta, P, Q), \forall i \in N$ 。每个 PowerFlowConv 层都包含一个类似于 [21] 的初始单跳消息传递步骤，用于提取图信息，包括边特征，以及在更高维特征空间上执行的 K 跳 TAGConv [22]。通过消息传递步骤，我们可以有效地利用边特征（即线路电阻和电抗）的信息更新编码后的图特征，从而使 TAGConv 能够在考虑边特征的同时，一次性高效地交换 K 跳邻域内的信息。通常，图卷积层（例如 TAGConv）不使用边特征的信息。然而，在 PF 公式中，线路特性起着重要作用。因此，通过消息传递，我们将边特征信息提炼到节点特征中，这些节点特征随后可供 TAGConv 使用，以提供高质量的 PF 近似。

更具体地说，消息传递方程 ∀i ∈ N 可描述为

$$\hat{x} = W \cdot \sigma \Big(W \sum_{j \in N} \langle x, x, x \rangle + b \Big) + b$$

（14）其中⟨x, x, x⟩是所述向量的串联向量，输入到一个两层 MLP 中，中间层使用 ReLU 激活函数。同样，权重矩阵和偏置向量 W、b、W 包含第 l 层 MLP 的可训练参数。

之后，第 l 层的中间图信号 eX 被计算为编码图特征 X 的总和，并且

处理后的消息传递图特征 X，即 X= Xare 此，TAGConv 层处理。然成新的编码图特征 Xas。

$$X = \sigma \Big(\sum_{k=0}^{K+1} (DAD)^k X W \Big) . \quad (15)$$

这里，(DAD) 表示基于对角度矩阵 D 的归一化邻接矩阵。如 [22] 所示，归一化邻接矩阵常用于图神经网络 (GNN) 中，以提供计算稳定性保证。然后，由第 l 层 PowerFlowConv 生成的编码图特征被传递到下一层，直到 l = L。最后，在最后一层使用消息传递步骤，而舍弃 TAGConv 步骤，如图 3 所示。

请注意，PowerFlowNet 对输入节点的排列具有等变性，并且在每个消息传递步骤中都利用了边特征。排列等变性是指无论输入节点的具体顺序如何，每个节点的输出都保持一致的特性。这种特殊的归纳倾向自然地源于 PF 问题本身，因为 PF 方程（参见第二节 B 部分）也具有排列等变性。此外，我们强调在消息传递过程的每个步骤中都融入边特征的重要性。这种融入至关重要，因为边特征是 PF 方程中的基本变量，对其进行考虑对于做出准确的预测至关重要。

3) 损失函数：选择合适的损失函数对于生成高质量的电力流近似模型至关重要。通常，现有的电力流机器学习方法使用纯粹基于物理的损失函数[18]，[20]。然而，由于电力流问题的特殊性，仅使用物理损失函数是不够的。因此，我们进行了进一步的实验，最终采用均方误差（MSE）来训练 PowerFlowNet，并开发了掩码 L2 损失作为更好的评估指标。具体而言，MSE 定义为：

$$MSE(y, \hat{x}) = ||y - \hat{x}|| \quad (16)$$

where $\boldsymbol{y}_i$ represents the real values of the features and $\hat{\boldsymbol{x}}_i$ are the predicted values of node i.

To get a more precise value of the actual error in predicting only the unknown features, we developed the Masked L2 loss. This loss function is similar to MSE but only calculates the error for the unknown features,

$$\text{MaskedL2}\left(\boldsymbol{y}_i, \hat{\boldsymbol{x}}_i\right) = ||\boldsymbol{m}_i \circ (\boldsymbol{y}_i - \hat{\boldsymbol{x}}_i)||_2^2. \tag{17}$$

Here, the $\circ$ is the element-wise multiplication operator, and $\boldsymbol{m}_i$ is the binary mask of node i indexing only the features of interest, as defined in III-B1.

IV. Experimental Evaluation

This section presents the experimental setup employed in this study, followed by a comprehensive evaluation aimed at elucidating the strengths and weaknesses of the PowerFlowNet model. To gain deeper insights into the efficacy of PowerFlowNet[2] as a GNN approach for PF approximation, the conducted ablation studies focus on four distinct aspects of the proposed approach: performance, interpretability, scalability, and architectural dependability.

A. Experimental Setup

The evaluation of PowerFlowNet's performance requires the consideration of the varied attributes present in each power network, encompassing differences in size and topology. To address this requirement comprehensively, three distinct power network cases are evaluated, namely IEEE case-14, IEEE case-118, and case 6470rte [25], based on their contrasting characteristics. The case with 14 nodes represents a minimal grid topology characterized by a limited number of connections, while the 118 case exhibits a more intricate configuration with a significantly larger number of nodes. Moreover, the 6470rte case accurately emulates the scale and intricacy of the French very high voltage and high voltage transmission network. It is important to note that realistic large power grids such as the 6470rte case are rarely investigated in PF problems because of the inability of most methods to seamlessly scale up. In contrast, we evaluate PowerFlowNet in this realistic large-scale network to fully show its scalability potential.

1) Dataset Generation: All aforementioned cases were modeled in PandaPower [26] and solved using the Newton-Raphson method. For each case, a total of over 30.000 distinct graph samples were generated by perturbing the default case files. The generation of each sample involved sampling from random uniform or normal distributions. Specifically, the resistance (r in Ω) and reactance (x in Ω) values of each line were uniformly selected within 80% and 120% of their original values. In the case of generators, the voltage magnitude (V^m) was uniformly set within the range of $[1.00, 1.05]$ per unit, while the initial active powers (P_g in MW) were sampled from a normal distribution $\mathcal{N}(P_g, 0.1|P_g|)$, where the mean corresponds to the original active power and the standard deviation is 10% of it. Lastly, for buses with loads, their active power (P in MW) and reactive power (Q in MVAR) were randomly sampled from normal distributions $\mathcal{N}(P, 0.1|P|)$ and $\mathcal{N}(Q, 0.1|Q|)$, respectively. These samples were then partitioned, allocating 50% for training, 20% for validation, and 30% for testing purposes.

2) Training & Evaluation: To train the models, the AdamW optimizer [27] was employed, using a learning rate of 0.001, in conjunction with the OneCycleLR scheduler [28]. The training process encompassed up to 2000 epochs (for large models) and 1000 epochs (for medium and small networks), with each epoch comprising a batch size of 128 samples. In our standard implementation, we use 4 PowerFlowConv layers ($L = 4$) of order 3 ($K = 3$) to ensure that the computation subgraph of every node covers the majority of the input graph. Additionally, we use 128 nodes for the hidden dimension and a dropout rate of 0.2 at the end of each layer to reduce overfitting. We train the models with MSE loss as defined in Equation 16. We also experimented with training with the physical loss and the mixed loss, defined in Equations 20 and 21 respectively. These additional results can be found in Section IV-E.

a) Physical Loss: This is a node-wise physical loss [20], defined by the unbalanced power at each node. This is a self-supervised loss function since it does not require the actual values of every node.

$$\text{Physical}\left(\hat{X}\right) = \frac{1}{|\mathcal{N}|} \sum_{i \in \mathcal{N}} ||\Delta P_i||_2^2 + ||\Delta Q_i||_2^2 \tag{18}$$

$$\Delta P_i = \hat{P}_i - \Re\left[\sum_{j \in \mathcal{N}(i)} \dot{V}_i \left(\frac{\dot{V}_j - \dot{V}_i}{r_{i,j} + j x_{i,j}}\right)^*\right] \tag{19}$$

$$\Delta Q_i = \hat{Q}_i - \Im\left[\sum_{j \in \mathcal{N}(i)} \dot{V}_i \left(\frac{\dot{V}_j - \dot{V}_i}{r_{i,j} + j x_{i,j}}\right)^*\right] \tag{20}$$

This loss function is purely based on the PF equations where $\mathcal{N}(i)$ is the set of neighbor nodes of node i, $\dot{V}_i = \hat{V}_i \angle \hat{\theta}_i$, $(\hat{V}_i, \hat{\theta}_i, \hat{P}_i, \hat{Q}_i)$ is the i-th row of $\hat{X}$, and $r_{i,j}$ and $x_{i,j}$ refer to the resistance and reactance of line (i, j), respectively.

b) Mixed Loss: Finally, we propose the Mixed loss that combines the physical and MSE loss to efficiently incorporate the PF equations into PowerFlowNet. . In detail, the Mixed loss for the whole graph with predicted feature matrix $\hat{X}$ and actual real feature value matrix Y is defined as:

$$\text{Mix}\left(Y, \hat{X}\right) = w \cdot \text{MSE}\left(Y, \hat{X}\right) + \tau \cdot (1 - w) \cdot \text{Physical}(\hat{X}) \tag{21}$$

where $w \in [0, 1]$ is a weight to balance the MSE and physical loss, and τ is a scaling factor bringing the physical loss to the same order of magnitude as the MSE loss.

c) Evaluation: During the evaluation, we compare the Masked L2 loss as a metric[3]. The execution times of each implemented method were examined using high-performance hardware configurations, including the "AMD RYZEN 7 5700X 8-Core" processor, "NVIDIA RTX 3060 TI 8GB"

[2]PowerFlowNet code, datasets, and trained models can be found at https://github.com/StavrosOrf/PoweFlowNet (will be public upon acceptance)*.

[3]We also experimented with training with Masked L2 loss, which resulted in almost the same performance as training with MSE loss. Therefore, we adopted the MSE loss because it gives a slightly better feature reconstruction.

其中 y 表示特征的真实值，ˆx 表示节点 i 的预测值。
为了更精确地评估仅预测未知特征时的实际误差，我们开发了掩码 L2 损失函数。该损失函数类似于均方误差（MSE），但仅计算未知特征的误差。

$$\text{MaskedL2}(y, \hat{x}) = ||m^{\circ}(y - \hat{x})||_{\circ} \quad (17)$$

这里，◦ 是逐元素乘法运算符，而 mis 是节点 i 的二进制掩码，仅索引感兴趣的特征，如 III-B1 中定义。

IV. EE

本节介绍本研究中使用的实验装置，并进行全面评估，旨在阐明 PowerFlowNet 模型的优势和劣势。为了更深入地了解 PowerFlowNet 作为一种用于 PF 近似的 GNN 方法的有效性，所进行的消融实验重点关注所提出方法的四个不同方面：性能、可解释性、可扩展性和架构可靠性。

A. 实验装置

评估 PowerFlowNet 的性能需要考虑每个电力网络中存在的各种属性，包括规模和拓扑结构的差异。为了全面满足这一要求，我们评估了三种不同的电力网络案例，分别是 IEEE case-14、IEEE case-118 和 case 6470rte [25]，这三种案例基于它们截然不同的特征。14 个节点的案例代表了一种连接数量有限的最小电网拓扑结构，而 118 个节点的案例则展现了一种节点数量显著更多的复杂配置。此外，6470rte 案例能够精确地模拟法国特高压和高压输电网络的规模和复杂性。值得注意的是，由于大多数方法无法无缝扩展，像 6470rte 这样真实的大型电力网络在功率因数问题中很少被研究。相比之下，我们在这个真实的大规模网络中评估 PowerFlowNet，以充分展示其可扩展性潜力。

1) 数据集生成：上述所有案例均在 PandaPower [26] 中建模，并使用牛顿-拉夫逊法求解。对于每个案例，通过扰动默认案例文件生成了超过 30,000 个不同的图样本。每个样本的生成均采用从均匀分布或正态分布中随机抽样的方法。具体而言，每条线路的电阻（r，单位为 Ω）和电抗（x，单位为 Ω）值在其原始值的 80% 和 120% 范围内均匀选取。对于发电机，电压幅值（V）均匀设置在标幺值 [1.00, 1.05] 的范围内，而初始有功功率（Pin，单位为 MW）则从正态分布 N(P, 0.1|P|) 中抽样，其中均值对应于原始有功功率，标准差为其 10%。最后，对于带负载的母线，其有功功率
功率（P，单位为兆瓦）和无功功率（Q，单位为兆乏）分别从正态分布 N(P, 0.1|P|) 和 N(Q, 0.1|Q|) 中随机抽取。然后将这些样本进行划分，其中 50% 用于训练，20% 用于验证，30% 用于测试。

2) 训练与评估：为了训练模型，我们采用了 AdamW 优化器 [27]，学习率为 0.001，并结合 OneCycleLR 调度器 [28]。训练过程最多包含 2000 个 epoch（大型模型）和 1000 个 epoch（中小型网络），每个 epoch 的批次大小为 128 个样本。在我们的标准实现中，我们使用 4 个 3 阶 PowerFlowConv 层 (L = 4, K = 3)，以确保每个节点的计算子图覆盖大部分输入图。此外，我们使用 128 个节点作为隐藏层，并在每层末尾设置 0.2 的 dropout 率以减少过拟合。我们使用公式 16 中定义的均方误差 (MSE) 损失函数训练模型。我们也尝试了分别使用公式 20 和 21 中定义的物理损失和混合损失进行训练。这些附加结果可在第四节 E 部分中找到。

a) 物理损耗：这是节点级的物理损耗[20]，由每个节点的功率不平衡程度定义。由于它不需要每个节点的实际值，因此是一种自监督损失函数。

$$\text{物理}(\hat{X}) = \frac{1}{|N|}\sum_{i\in N} ||\Delta P|| + ||\Delta Q|| \quad (18)$$

$$\Delta P = \hat{P} - \Re\left(V \sum_{j\in N(i)} V^{*} \cdot \frac{1}{r + jx}\right) \quad (19)$$

$$\Delta Q = \hat{Q} - \Im\left(V \sum_{j\in N(i)} V^{*} \cdot \frac{1}{r + jx}\right) \quad (20)$$

该损耗函数完全基于 PF 方程，其中 N(i)是节点 i 的邻居节点集合，·V=ˆV∠ˆθ，(ˆV, ˆθ, ˆP, ˆQ)是ˆX 的第 i 行，而 x 和 y 分别指线路(i, j)的电阻和电抗。

b) 混合损失：最后，我们提出了一种混合损失，它结合了物理损失和均方误差损失，以有效地将 PF 方程融入 PowerFlowNet 中。具体来说，对于整个图，预测特征矩阵为ˆX，实际真实特征值矩阵为 Y，混合损失定义如下：

$$\text{混合}(\hat{X}, Y) = w \cdot \text{MSE}(Y, \hat{X}) + \tau \cdot (1-w) \cdot \text{物理}(\hat{X}) \quad (21)$$

其中 w ∈ [0, 1] 是平衡 MSE 和物理损失的权重，τ 是缩放因子，使物理损失与 MSE 损失处于同一数量级。

c) 评估：在评估过程中，我们以掩码 L2 损失作为衡量指标。我们使用高性能硬件配置（包括“AMD RYZEN 7 5700X 8 核”处理器和“NVIDIA RTX 3060 TI 8GB”）测试了每种已实现方法的执行时间。

[2] PowerFlowNet 代码、数据集和训练好的模型可以在 https://github.com/StavrosOrf/PowerFlowNet 找到（接受后将公开）*。

[2] 我们也尝试了使用掩码 L2 损失进行训练，结果与使用 MSE 损失的训练效果几乎相同。因此，我们最终采用了 MSE 损失，因为它能提供略好一些的特征重构效果。

TABLE I
PERFORMANCE COMPARISON OF POWERFLOWNET.

Case	14		118		6470rte	
Algorithms	Masked L2 Loss	Time (ms)	Masked L2 Loss	Time (ms)	Masked L2 Loss	Time (ms)
Newton-Raphson	≈ 0	17.0	≈ 0	20.0	≈ 0	580.0
DC Power Flow	45.74	8.0	99.87	10.0	510.5	30.0
Tikhonov Reg.	2.838	0.4	1.916	0.4	2.934	6100
3-Layer MLP	0.034	0.6	0.462	0.6	0.590	0.6
3-Layer GCN	0.239	1.0	1.257	1.0	1.957	2.0
PowerFlowNet	**0.002**	4.0	**0.022**	4.0	**0.303**	4.0

graphics card, and 32GB RAM, ensuring accurate and efficient measurements. Notably, it is important to emphasize the reproducibility of the experiments, as the availability of datasets, the PowerFlowNet framework, and the trained models provided ensure the feasibility of replicating the study's outcomes and procedures.

B. Performance Comparison

In this comparison study, PowerFlowNet is evaluated against other baseline methods for PF approximation across three distinct cases, as outlined in Table I. The performance assessment of each approach is conducted based on accuracy, quantified by employing the Masked L2 Loss, as well as the execution time on the same computing machine. The datasets used for evaluation were generated using the Newton-Raphson method, recognized for its accuracy in PF analysis, resulting in zero loss for these particular cases. As expected, the Newton-Raphson method has increased execution time as the scale of the PF problem grows. The second baseline method considered is the DCPF approach, which relaxes the nonlinear problem to a linear one, thereby serving as an upper-performance boundary for the comparison. However, the DCPF only calculates each bus' voltage angle and active power since it assumes the voltage magnitude to be constant across the grid. PowerFlowNet exhibits remarkable performance superiority in terms of loss, surpassing the DCPF method by several orders of magnitude, depending on the case. Additionally, our proposed approach demonstrates significantly reduced execution time compared to the Newton-Raphson method, with speed improvements of $4\times$ in the 14-node case, $5\times$ in the 118-node case, and an impressive $145\times$ in the 6470rte case. Notably, owing to the GNN architecture of PowerFlowNet, the execution time remains consistently low across different cases. This inherent scalability characteristic of GNNs represents a significant advantage over traditional methods, enabling the real-time approximation of PF in extensive power networks. Note that the computation time of Newton-Raphson is acceptable ($\approx$ 0.6s) when computing a small number of cases but might not be enough when thousands of computations are needed in tasks such as planning where the uncertainty of distributed energy resources is introduced. For example, the Newton-Raphson method would take around 20 minutes to calculate 2000 PF cases for the 6470rte grid, while PowerFlowNet would only need around 1.3 minutes.

Furthermore, a comprehensive evaluation of alternative traditional ML approaches was conducted to ascertain the unique benefits of utilizing PowerFlowNet. Initially, the Tikhonov regularizer [29], a method designed to enforce smoothness while considering the underlying graph structure, was assessed. However, as indicated in Table I, its performance consistently fell behind the other ML methods in terms of loss across all cases, as well as in execution time within the context of large-scale power networks. Subsequently, a three-layer Multilayer Perceptron and a simplified GNN consisting of merely three layers of GCNs [15] were evaluated. Although neither approach surpassed PowerFlowNet in terms of loss (MLP was $16\times$ worse and the GCN $12\times$ worse in the 14-case), it is noteworthy that the MLP exhibited a substantially faster execution time, differing by an order of magnitude ($\approx 6.5\times$ faster).

TABLE II
AVERAGE ABSOLUTE DENORMALIZED ERROR

Case	V^m (p.u.)	θ (*deg*)	P (MW)	Q (Mvar)
14	0.0008 ± 0.0006	0.09 ± 0.07	0.02 ± 0.03	0.13 ± 0.16
118	0.0022 ± 0.0018	0.64 ± 0.54	0.18 ± 0.36	1.20 ± 1.47
6470rte	0.0072 ± 0.0061	10.2 ± 7.89	0.23 ± 0.42	0.12 ± 6.46

C. Denormalized Error Distributions

After showcasing the performance of PowerFlowNet in three different power networks, each with unique characteristics, it is important to delve deeper into the results and visualize the denormalized error distributions. This analytical approach allows us to gain valuable insights into the areas where more accurate predictions can be achieved, as well as the conditions under which these improvements are most evident. Such insights hold practical significance for power systems operators, enabling them to leverage PowerFlowNet effectively in real-world scenarios involving network operation and strategic planning. Table II shows the average error distribution for every one of the four predicted values[4].

Notice that the proposed model achieves precise predictions of p.u. voltage magnitude for all cases, exhibiting an average error of 8×10^{-4} p.u. in the 14-node scenario and 72×10^{-4} in the notably more intricate 6470rte case, all while maintaining a consistently low standard deviation across each case. Additionally, the average absolute error remains minimal for both active and reactive power with respect to the network's actual demand and load, with an actual error of 20 kW for active power and 130 kvar for reactive power in the best-case scenario. Nevertheless, it is important to note a significant rise in the standard deviation of predicted reactive power error, particularly in the 6470rte case, where it reaches as high as 6460 kvar. Similarly, the voltage angle prediction error is barely noticeable in the 14-node case, being only 0.09 degrees, but increases significantly to 10.2 degrees in the 6470rte case. This occurs because the range of the voltage angles across larger networks network, i.e., the 6470rte case, is much greater.

Furthermore, we can observe the per-node denormalized error distribution for every case in Fig. 4. Notice that the

[4]The error distributions include only the predictions of the unknown values.

表一

PFN 性能比较。

案件	14		118		6470rte	
算法	蒙面人 L2 陆地	时间（多发性硬化症）	蒙面人 L2 陆地	时间（多发性硬化症）	蒙面人 L2 陆地	时间（多发性硬化症）
牛顿-拉夫逊法	≈ 0	17.0	≈ 0	20.0	≈ 0	580.0
直流功率流	45.74	8.0	99.87	10.0	510.5	30.0
吉洪诺夫规则	2.838	0.4	1.916	0.4	2.934	6100
三层 MLP	0.034	0.6	0.462	0.6	0.590	0.6
三层 GCN	0.239	1.0	1.257	1.0	1.957	2.0
PowerFlowNet	0.002	4.0	0.022	4.0	0.303	4.0

显卡和 32GB 内存确保了测量的准确性和高效性。值得注意的是，实验的可重复性至关重要，因为所提供的数据集、PowerFlowNet 框架和训练模型保证了研究结果和流程的可复现性。

B. 性能比较

在本对比研究中，PowerFlowNet 在表 I 所示的三种不同情况下，与其他基准方法在功率因数近似方面进行了评估。每种方法的性能评估基于精度（通过掩码 L2 损耗量化）以及在同一台计算机上的执行时间。用于评估的数据集由牛顿-拉夫逊法生成，该方法因其在功率因数分析中的精度而著称，在这些特定情况下实现了零损耗。正如预期的那样，随着功率因数问题规模的增大，牛顿-拉夫逊法的执行时间也随之增加。考虑的第二个基准方法是 DCPF 方法，它将非线性问题简化为线性问题，从而作为比较的性能上限。然而，由于 DCPF 假设电网电压幅值恒定，因此它仅计算每个母线的电压相角和有功功率。PowerFlowNet 在损耗方面表现出显著的性能优势，根据具体情况，其性能比 DCPF 方法高出几个数量级。此外，与牛顿-拉夫逊法相比，我们提出的方法显著缩短了执行时间，在 14 节点的情况下速度提升了 4 倍，在 118 节点的情况下提升了 5 倍，在 6470rte 的情况下更是提升了惊人的 145 倍。值得注意的是，由于 PowerFlowNet 采用了图神经网络（GNN）架构，其执行时间在不同情况下始终保持较低水平。GNN 固有的这种可扩展性使其相比传统方法具有显著优势，能够对大型电力网络中的功率因数进行实时近似计算。 需要注意的是，牛顿-拉夫逊法在计算少量算例时计算时间尚可接受（≈ 0.6 秒），但在需要进行数千次计算的任务（例如分布式能源资源不确定性较大的规划任务）中，其计算时间可能不足。例如，牛顿-拉夫逊法计算 6470rte 电网的 2000 个功率因数算例大约需要 20 分钟，而 PowerFlowNet 仅需约 1.3 分钟。

此外，我们还对其他传统机器学习方法进行了全面评估，以确定其独特性。

表二

主意

案件	在（pu）	θ（你）	P（兆瓦）Q（兆乏）	
14	0.0008 ± 0.0006	0.09 ± 0.07	0.02 ± 0.03	0.13 ± 0.16
0.0022±0.0018	0.64±0.54	0.18±0.36	1.20±1.47	6470rte
0.0072±0.0061	10.2±7.89	0.23±0.42	0.12±6.46	

利用 PowerFlowNet 的优势。首先，我们评估了 Tikhonov 正则化器 [29]，该方法旨在考虑底层图结构的同时强制执行平滑性。然而，如表 I 所示，在所有情况下，其损失值均落后于其他机器学习方法，并且在大规模电力网络场景下的执行时间也较慢。随后，我们评估了三层多层感知器 (MLP) 和仅由三层图卷积网络 (GCN) 组成的简化图神经网络 (GNN) [15]。尽管这两种方法在损失值方面均未超越 PowerFlowNet（在 14 层场景下，MLP 的损失值比 GCN 低 16 倍，GCN 低 12 倍），但值得注意的是，MLP 的执行时间显著更快，快了一个数量级（约 6.5 倍）。

C. 非归一化误差分布

在展示了 PowerFlowNet 在三个具有独特特征的电力网络中的性能之后，深入分析结果并可视化非归一化误差分布至关重要。这种分析方法使我们能够深入了解哪些领域可以实现更精确的预测，以及这些改进最为显著的条件。这些见解对电力系统运营商具有重要的实际意义，使他们能够在涉及网络运行和战略规划的实际场景中有效地利用 PowerFlowNet。表 II 显示了四个预测值中每个值的平均误差分布。

值得注意的是，所提出的模型在所有情况下都能精确预测标幺值电压幅值，在 14 节点场景下平均误差为 8 × 10 标幺值，在更为复杂的 6470rte 场景下平均误差为 72 × 10 标幺值，且在所有情况下均保持较低的标准偏差。此外，相对于网络的实际需求和负荷，有功功率和无功功率的平均绝对误差均保持在极小水平，最佳情况下有功功率的实际误差为 20 kW，无功功率的实际误差为 130 kvar。然而，需要注意的是，预测无功功率误差的标准偏差显著增加，尤其是在 6470rte 场景下，高达 6460 kvar。同样，电压角度预测误差在 14 节点场景下几乎可以忽略不计，仅为 0.09 度，但在 6470rte 场景下显著增加至 10.2 度。这是因为大型网络（例如 6470rte 情况）的电压角度范围要大得多。

此外，我们可以从图 4 中观察到每种情况下每个节点的非归一化误差分布。请注意，

[4] 误差分布仅包含未知值的预测值。

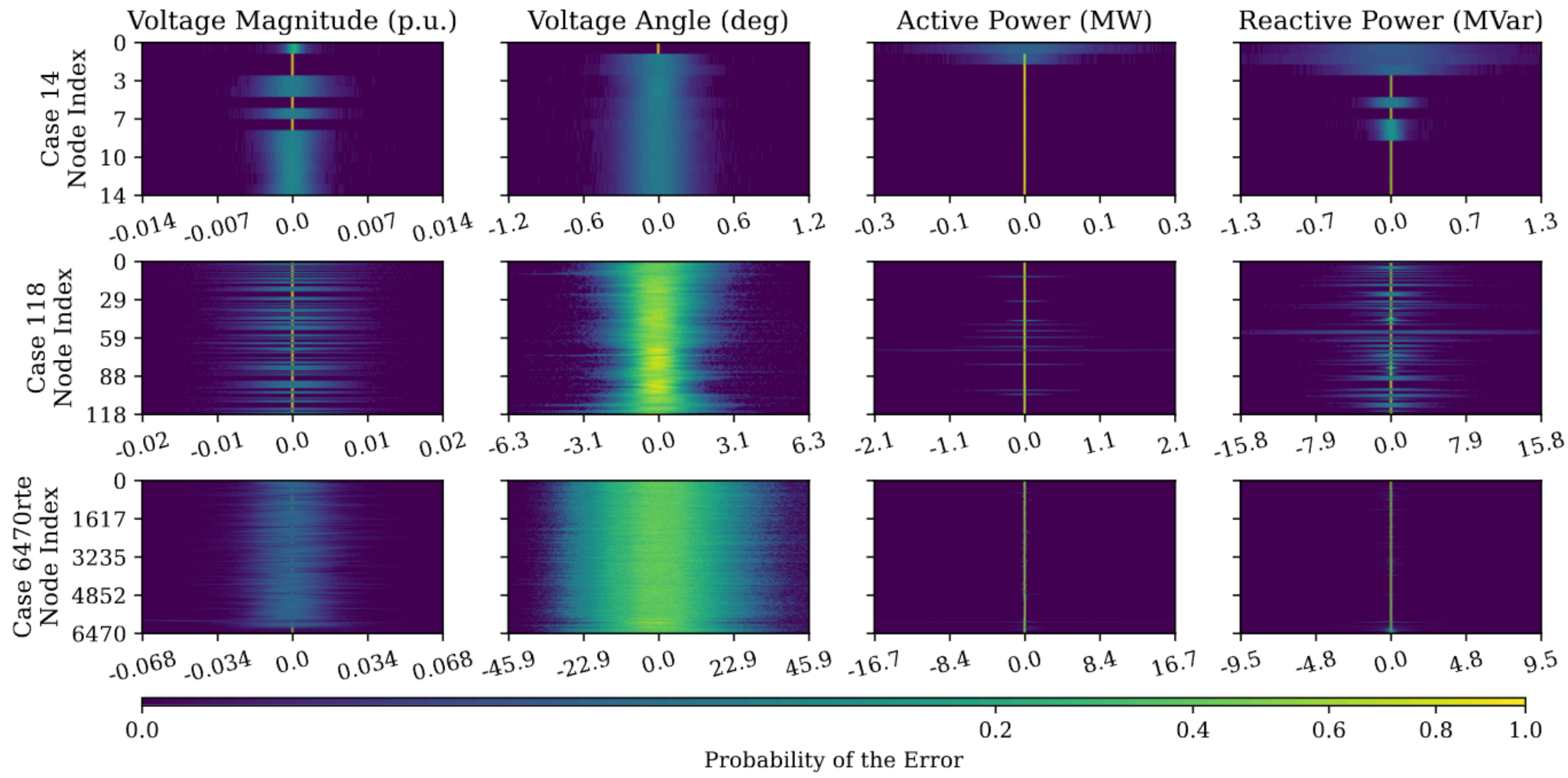


Fig. 4. Probability density function of the actual not-normalized error for every node in the test dataset. The brighter the color, the highest the probability of the prediction error of PowerFlowNet being in that region.

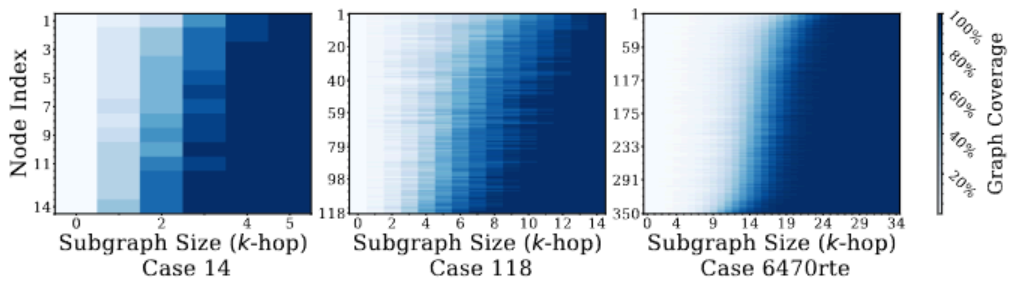


Fig. 5. Illustration of graph coverage defined as the number of nodes included in a subgraph, plotted per K-hop size and sorted from slowest to fastest growth for every node.

prediction error for nodes where we already know the value is practically zero (indicated by the vertical yellow line). Although the average voltage angle error is higher for the 6470rte case (10.2 degrees as shown in Table II), the average voltage magnitude error is in the same range (less than 0.008 p.u.) as in the smaller networks, ensuring that the developed PowerFlowNet is accurate enough for fast checking of operational constraints for large numbers of scenarios regardless of network size.

D. Interpretability Study

To gain insights into the locality of information used by PowerFlowNet to make predictions, we aimed to investigate potential affordances in using subgraphs as a more scalable and generalizable approach to tackle the challenges posed by the complex PF prediction task.

We examined K-hop subgraphs across all three cases, where, for reference, a PowerFlowNet model with a kernel size of $K = 3$ and four layers of TAGConv ($L = 4$), a straightforward computation yields an effective receptive field spanning 12 nodes. In practice, the size of these receptive fields, measured in the number of nodes, will vary based on the specific graph topology. To better understand this topological effect, we looked at the speed at which K-hop subgraphs grow from each node in the graph, also defined as graph coverage. In detail, for each node, we made K-hop subgraphs for K between 1 and the graph diameter and measured the subgraph size after each hop, as shown in Fig. 5. This indicates the distribution of receptive field sizes seen during training; interestingly, it appears to form a Gaussian shape, particularly visible in the largest graph in the dataset with 6470 nodes. As expected the larger the graph size the more hopes are required to cover the whole graph. For example, 4 hops are required in the 14-node over 6 hops in the 118-node case, and 11 to 25 hops in the 6470rte case.

Afterward, those subgraphs were each passed through the PowerFlowNet model, and the Masked L2 loss was calculated only on the node from which the K-hop neighborhood was generated. This showed us the actual loss distribution among nodes depending on how far a K-hop neighborhood can reach. The detailed results are illustrated in Fig. 6. We noticed that this loss distribution has a different shape from the subgraph growth figure, and indeed there is variation in which nodes are more or less affected by a decrease in subgraph size.

Interestingly, we can observe that only around 3 hops are required to obtain the minimum loss for most of the nodes, experimentally proving the reason TAGConv of order $K = 3$ can achieve high-quality results. This also suggests that for most buses (nodes) in a power network, we could make PowerFlowNet predict the PF values even faster by only looking at smaller connected parts.

Finally, we plot the average loss across nodes in Fig. 7 and discover that neighborhood hop counts well below the effective

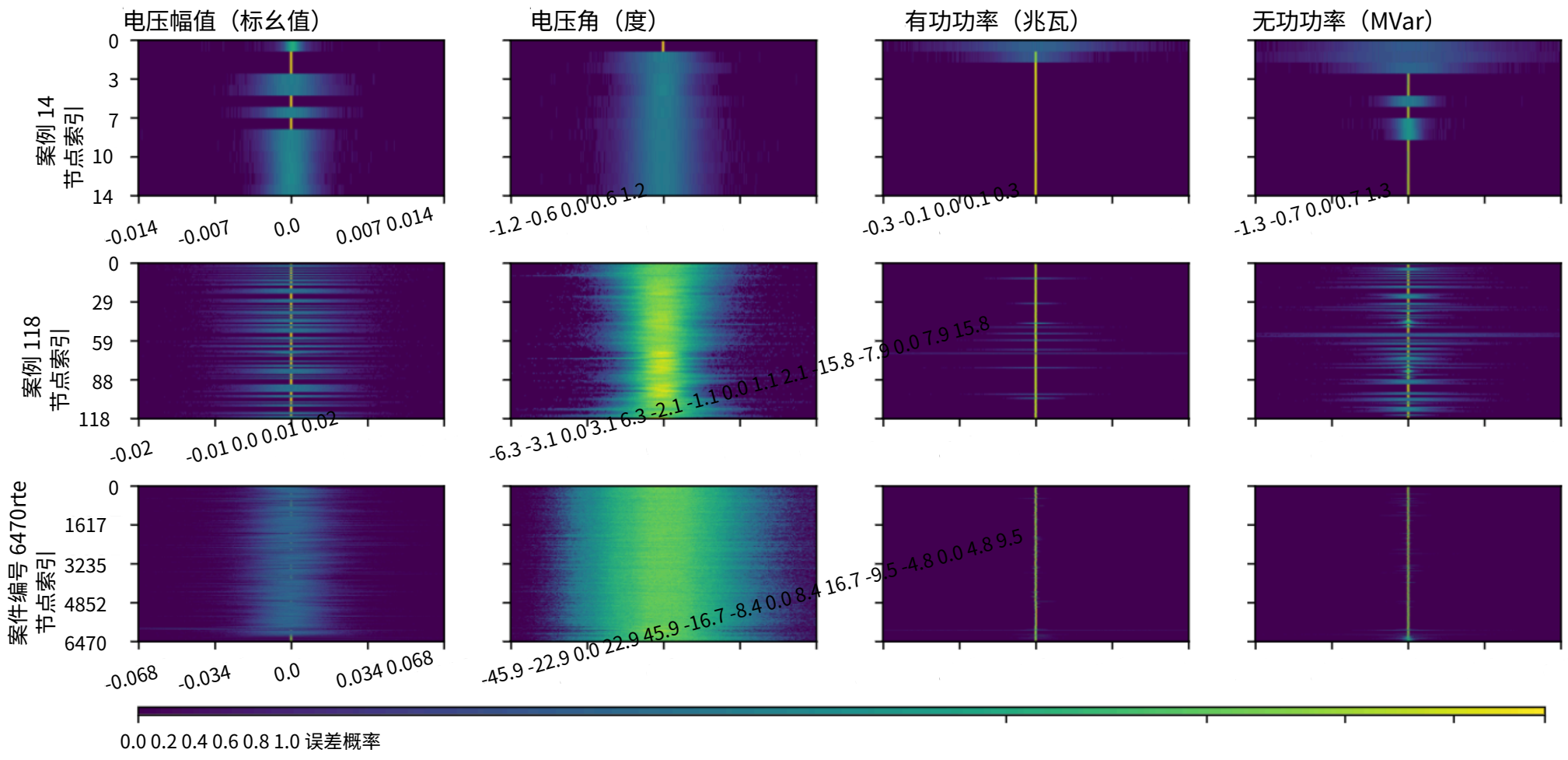


图 4. 测试数据集中每个节点的实际未归一化误差的概率密度函数。颜色越亮，表示 PowerFlowNet 的预测误差位于该区域的概率越高。

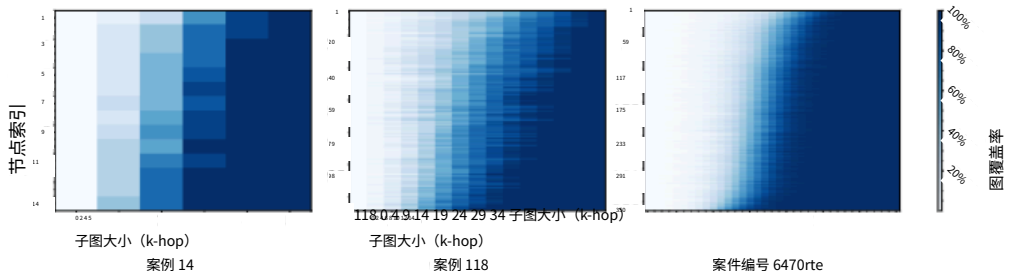


图 5. 图覆盖率示意图，定义为子图中包含的节点数，按 K 跳大小绘制，并按每个节点的增长速度从慢到快排序。

对于已知值几乎为零的节点（由黄色竖线标示），预测误差较小。虽然 6470rte 案例的平均电压角度误差较高（如表 II 所示为 10.2 度），但平均电压幅值误差与较小规模的网络处于同一量级（小于 0.008 pu），这确保了所开发的 PowerFlowNet 模型足够精确，可以快速检查大量场景下的运行约束，而无需考虑网络规模。

D. 可解释性研究

为了深入了解 PowerFlowNet 用于进行预测的信息的局部性，我们旨在研究使用子图作为更具可扩展性和通用性的方法的潜在优势，以应对复杂的 PF 预测任务带来的挑战。

我们考察了所有三种情况下的 K 跳子图。作为参考，对于卷积核大小为 K = 3、具有四层 TAGConv (L = 4) 的 PowerFlowNet 模型，通过简单的计算可得出有效感受野跨越 12 个节点。实际上，这些感受野的大小……

以节点数衡量的覆盖率会根据具体的图拓扑结构而变化。为了更好地理解这种拓扑效应，我们研究了从图中每个节点生成 K 跳子图的速度，也称为图覆盖率。具体来说，对于每个节点，我们生成了 K 跳子图，K 值介于 1 和图直径之间，并测量了每次跳跃后子图的大小，如图 5 所示。这显示了训练过程中感受野大小的分布；有趣的是，它似乎呈现出高斯形状，在数据集中最大的图（包含 6470 个节点）中尤为明显。正如预期的那样，图越大，覆盖整个图所需的跳跃次数就越多。例如，14 个节点的图需要 4 次跳跃，而 118 个节点的图则需要 6 次跳跃，在 6470 个节点的图中则需要 11 到 25 次跳跃。

随后，我们将这些子图分别输入到 PowerFlowNet 模型中，并仅在生成 K 跳邻域的节点上计算掩码 L2 损失。这展示了节点间实际的损失分布，该分布取决于 K 跳邻域的覆盖范围。详细结果如图 6 所示。我们注意到，该损失分布的形状与子图增长图的形状不同，并且确实存在节点受子图尺寸减小影响程度不同的情况。

有趣的是，我们可以观察到，对于大多数节点而言，仅需约 3 跳即可获得最小损耗，这从实验上证明了阶数 K = 3 的 TAGConv 能够取得高质量结果的原因。这也表明，对于电力网络中的大多数母线（节点），我们可以通过仅关注较小的连接部分，使 PowerFlowNet 更快地预测功率因数 (PF) 值。

最后，我们在图 7 中绘制了节点间的平均损失，并发现邻域跳数远低于有效损失。

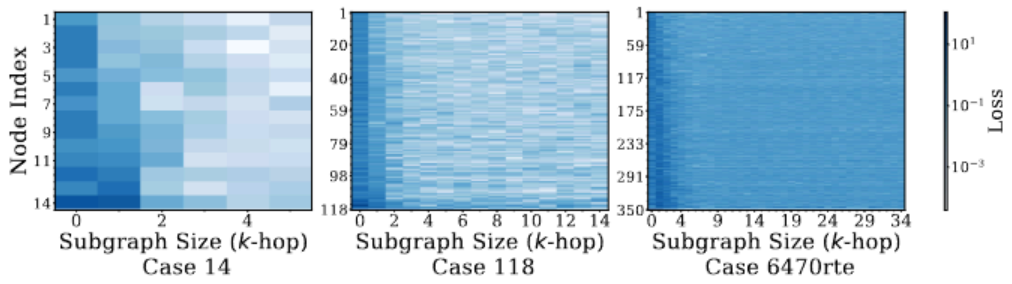


Fig. 6. Masked L2 loss on the central node of a k-hop subgraph for each node and k, sorted by the total node loss.

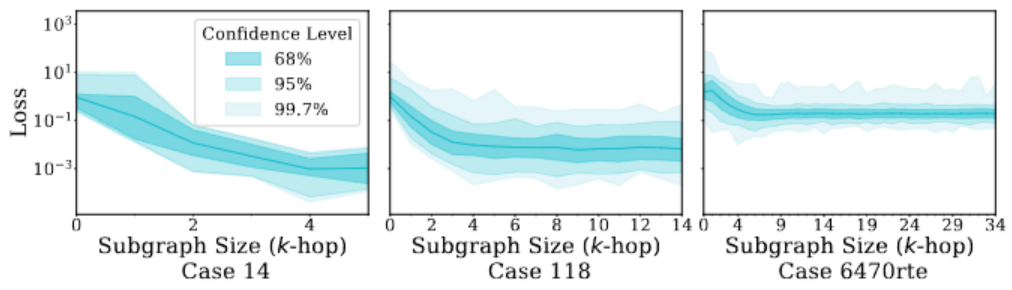


Fig. 7. Average Masked L2 loss on the central nodes of a k-hop subgraph for different numbers of k.

TABLE III
MASKED L2 LOSS OF SCALED MODELS ON THE 118 CASE.

Model	# Params	Trained with		
		MSE	Physical	Mixed
PowerFlowNet (Small)	32k	0.079	0.757	0.109
PowerFlowNet	357k	0.022	0.667	0.057
PowerFlowNet (Large)	7375k	**0.002**	**0.628**	**0.004**

receptive field's hop count are enough to make predictions practically identical to those when using the full graph. This indicates that local predictions in a large network are feasible. Most importantly, we argue that training on larger graphs could be made more efficiently by sampling subgraphs across different graphs available in the dataset, so as to make sure that stochastic gradient descent steps are performed using batches that capture more of the variation available in the dataset. Subsequently, this approach has the potential to augment the scalability aspects of PowerFlowNet.

E. Model Scalability Study

We demonstrate that our model has valuable scalability features; as we investigate how PowerFlowNet's performance scales with the model size, the size of the training dataset, and the loss function used.

Based on the standard setting PowerFlowNet (Medium) with $L = 4, h = 128$, we created two variants by modifying the total number of the model's trainable parameters by one order of magnitude as shown in Table III. In detail, we created PowerFlowNet (Small) with $L = 2, h = 64$ and PowerFlowNet (Large) with $L = 5, h = 512$. Table III shows the relation between the model size and the Masked L2 loss when trained with different loss functions for the 118-node case. We note that the performance scales very well with the model size. Most importantly, from the medium to large model, the Masked L2 loss is almost one order of magnitude lower, signifying the learning potential of PowerFlowNet.

The model performance does not differ greatly when trained with physical loss (as used in [20]). This means learning with physical loss is challenging regardless of the model's capability. However, we notice that the Masked L2 performance difference between training with MSE and mixed loss shrinks as we move to the large model. This suggests that this capability gain allows us to optimize both the MSE and the physical loss, thus making precise and physics-conforming predictions.

TABLE IV
PERFORMANCE EVALUATION FOR VARYING TRAINING CASES.

Eval. Case		118			14		
Eval. Metric		M. L2	MSE	Phys.	M. L2	MSE	Phys.
Train. Case	118+14 (L)	**0.0023**	**0.0011**	**4.9676**	**0.0014**	**0.0006**	**0.1336**
	118 (L)	0.0029	0.0014	5.4080	6.1998	2.5947	40.942
	118 (M)	0.0220	0.0105	58.9221	3.7860	3.7860	45.426

TABLE V
POWERFLOWNET COMPONENT SIGNIFICANCE ANALYSIS

Case	Full Model	Model 1-Layer	No Message Passing (MP)	1-Layer & No MP
14	**0.002**	0.11	0.78	0.08
118	**0.022**	0.38	1.01	0.24
6470rte	**0.303**	1.12	1.91	0.88

To further test PowerFlowNet's capability, we trained the large model on a mixed dataset composed of the 14 and the 118 cases. We unified the training loss function with MSE loss and evaluated the trained models using different metrics (MSE, Masked L2, and physical loss) on each of the two cases individually. The results are given in Table IV. After comparing with training large (L) and medium (M) models on the 118-node case, we noticed that even without explicitly bringing physical loss in training, the large models automatically learn the underlying physics, yielding more physics-conformed predictions. Large models are around 10× better in terms of loss when trained and evaluated on the same case. When we augment the 118-node case training set with the 14-node case, the Large PowerFlowNet model performs even better than when trained only in the 118-node case (0.0006 less Masked L2 loss), achieving an excelling accuracy in the 14-node case (0.0006 less Masked L2 loss than just training in the 14 case). This suggests that the proposed large PowerFlowNet model can operate on multiple graphs and learn the underlying physics without sacrificing accuracy.

F. Architectural Ablation Study

Finally, architectural ablation experiments were carried out to demonstrate the vitality of PowerFlowNet's unique structures through component removal. Therefore, the experiments were designed to highlight how each PowerFlowNet component contributes to generating high-quality predictions. Table V depicts the results of these experiments. Table V is split into four variations of PowerFlowNet: the full model, PowerFlowNet with $L = 1$, PowerFlowNet without message passing, and PowerFlowNet with $L = 1$ and without message passing. Then, we can observe how the model performs in terms of Masked L2 loss in all different scenarios.

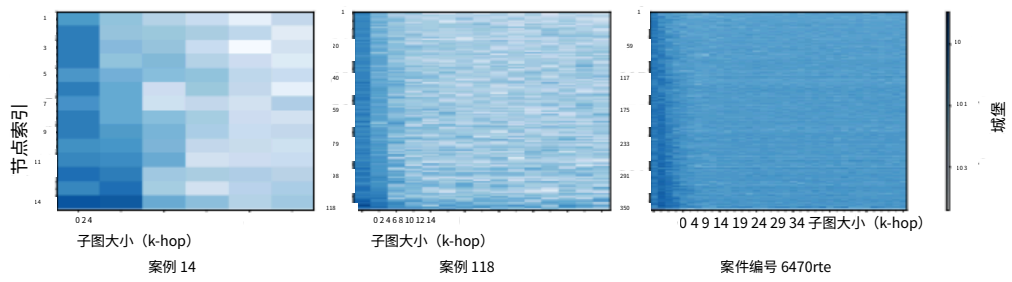


图 6. k 跳子图中心节点上的掩码 L2 损失，针对每个节点和 k，按节点总损失排序。

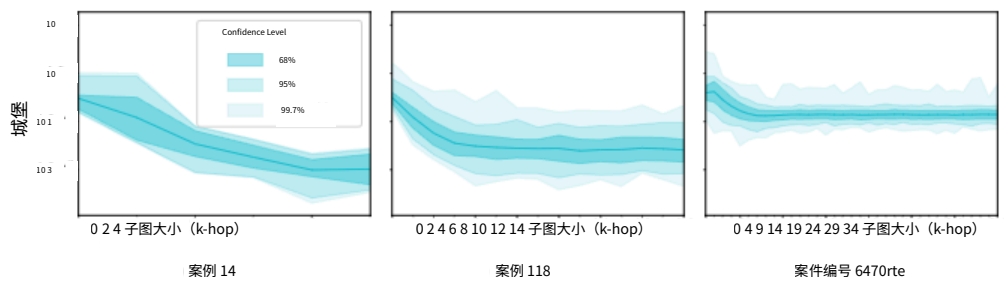


图 7. 不同 k 值下 k 跳子图中心节点的平均掩码 L2 损失。

表三

ML2 损失函数在 118 个缩放模型上。

模型	#参数	接受过培训	
		MSE	物理混合
PowerFlowNet（小型）	32k	0.079	0.757
0.109 PowerFlowNet	357k	0.022	
0.667	0.057	PowerFlowNet（大型）	7375k
0.002	0.628	0.004	

感受野的跳数足以使预测结果与使用完整图时的结果几乎完全相同。这表明在大型网络中进行局部预测是可行的。更重要的是，我们认为，通过在数据集中不同图上采样子图，可以更有效地在大图上进行训练，从而确保随机梯度下降步骤使用能够捕捉数据集中更多变异的批次执行。因此，这种方法有可能增强 PowerFlowNet 的可扩展性。

E. 模型可扩展性研究

我们证明了我们的模型具有有价值的可扩展性特征；我们研究了 PowerFlowNet 的性能如何随模型大小、训练数据集的大小以及所使用的损失函数而扩展。

基于标准设置 PowerFlowNet (Medium)（L = 4，h = 128），我们通过将模型的可训练参数总数增加一个数量级，创建了两个变体，如表 III 所示。具体而言，我们创建了 L = 2，h = 64 的 PowerFlowNet (Small) 和 L = 5，h = 512 的 PowerFlowNet (Large)。表 III 展示了在 118 个节点的情况下，使用不同损失函数训练时，模型规模与 Masked L2 损失之间的关系。我们注意到，性能随模型规模的增加而显著提升。更重要的是，从中等规模到大型模型，Masked L2 损失降低了近一个数量级，这表明 PowerFlowNet 具有强大的学习潜力。

使用物理损失（如[20]中所用）训练模型时，模型性能并没有显著差异。这意味着学习

表四

针对不同训练案例的绩效评估。

评估案例	118			14		
评估指标	M. L2	MSE	物理	M. L2	MSE	物理
118+14 (L)	0.0023	0.0011	4.9676	0.0014	0.0006	0.1336
列车。案例	118	(L)	0.0029	0.0014	5.4080	6.1998	2.5947	40.942
	118	(M)	0.0220	0.0105	58.9221	3.7860	3.7860	45.426

表 V

PFNET 成分重要性分析

案件	满的 模型	模型 单层	无消息 通过（MP）	单层 & 没有议员
14	0.002	0.11	0.78	0.08
118	0.022	0.38	1.01	0.24
6470rte	0.303	1.12	1.91	0.88

无论模型的性能如何，使用物理损失进行训练都是一项挑战。然而，我们注意到，随着模型规模的增大，使用均方误差（MSE）和混合损失进行训练时，掩码 L2 的性能差异逐渐缩小。这表明，这种性能提升使我们能够同时优化均方误差和物理损失，从而做出精确且符合物理规律的预测。

为了进一步测试 PowerFlowNet 的性能，我们使用包含 14 个节点和 118 个节点两种情况的混合数据集训练了大型模型。我们将训练损失函数统一为均方误差 (MSE) 损失，并分别在两种情况下使用不同的指标（MSE、Masked L2 损失和物理损失）评估了训练好的模型。结果如表 IV 所示。与在 118 个节点的情况下训练大型 (L) 和中型 (M) 模型相比，我们发现，即使没有在训练中显式引入物理损失，大型模型也能自动学习底层物理规律，从而产生更符合物理规律的预测结果。在相同的情况下训练和评估时，大型模型的损失比中型模型低约 10 倍。当我们用 14 节点的情况来扩充 118 节点的情况训练集时，大型 PowerFlowNet 模型性能甚至优于仅在 118 节点情况下训练的情况（Masked L2 损失降低了 0.0006），并且在 14 节点情况下也取得了卓越的准确率（Masked L2 损失比仅在 14 节点情况下训练的情况降低了 0.0006）。这表明，所提出的大型 PowerFlowNet 模型可以在多个图上运行，并在不牺牲准确率的情况下学习底层物理特性。

F. 建筑消融研究

最后，我们进行了架构消融实验，通过移除组件来验证 PowerFlowNet 独特结构的有效性。这些实验旨在突出 PowerFlowNet 的每个组件如何为生成高质量预测做出贡献。表 V 展示了这些实验的结果。表 V 将 PowerFlowNet 分为四种变体：完整模型、L=1 的 PowerFlowNet、不进行消息传递的 PowerFlowNet 以及 L=1 且不进行消息传递的 PowerFlowNet。然后，我们可以观察模型在各种不同场景下，基于 Masked L2 损失的性能表现。

As observed, none of the PowerFlowNet's variations performed as well as the complete model. In the 1-layer case, it is visible that the model fails to capture distant node dependencies, resulting in up to $55\times$ higher error. Similarly, the model performs poorly (up to $390\times$ higher error) in the no message passing case since the edge features are completely ignored, highlighting the importance of incorporating the edge features. On the other hand, the 1-layer model without message passing performs the best (up to $40\times$) since its functionality is similar to stacked MLPs. Consequently, this analysis shows that every part of PowerFlowNet is important in achieving high-quality PF approximations.

V. Conclusions

In this paper, we presented PowerFlowNet, a novel data-driven algorithm that capitalized on the efficiency and capabilities of GNN operations applied to the power network's topology, leading to a precise approximation of the PF. Specifically, our approach transformed the traditional PF problem into a GNN node-regression task by representing buses as nodes and transmission lines as edges while preserving network connectivity. Our model featured a distinctive configuration involving a mask encoder in conjunction with a sequence of our proposed PowerFlowConv layers, designed to aggregate features across the entire graph and inherently learn the dynamics of the underlying PF. The results verified PowerFlowNet's ability to generate high-quality solutions for a diverse set of network cases including the 6470rte very large-scale power network of France. Most importantly, our method produces accurate PF predictions at a fraction of the time of traditional solvers, regardless of the scale of the network, outperforming the well-established Newton-Raphson method in terms of execution time while having closely comparable results. Ultimately, our ablation studies reveal the robust aspects and potential limitations of our approach, thereby enabling the versatile deployment of PowerFlowNet across a wide array of real-world power system operation and planning scenarios.

References

[1] M. Albadi, "Power flow analysis," in *Computational Models in Engineering*, K. Volkov, Ed. Rijeka: IntechOpen, 2019, ch. 5.

[2] V. M. da Costa, N. Martins, and J. L. R. Pereira, "Developments in the newton raphson power flow formulation based on current injections," *IEEE Trans. Power Syst.*, vol. 14, pp. 1320–1326, 1999.

[3] A. Eltamaly and A. A. Elghaffar, "Load flow analysis by gauss-seidel method; a survey," *International Journal of Mechatronics, Electrical and Computer Technology (IJMEC) 2305-0543, 2411-6173*, vol. x, 10 2017.

[4] H. Seifi and M. Sepasian, *Electric Power System Planning: Issues, Algorithms and Solutions*. Springer, 01 2011, vol. 49.

[5] Y. Liu, N. Zhang, Y. Wang, J. Yang, and C. Kang, "Data-driven power flow linearization: A regression approach," *IEEE Trans. on Smart Grid*, vol. 10, no. 3, pp. 2569–2580, 5 2019.

[6] L. Guo, Y. Zhang, X. Li, Z. Wang, Y. Liu, L. Bai, and C. Wang, "Data-Driven Power Flow Calculation Method: A Lifting Dimension Linear Regression Approach," *IEEE Trans. on Power Syst.*, vol. 37, no. 3, pp. 1798–1808, 5 2022.

[7] J. Chen, W. Wu, and L. A. Roald, "Data-driven piecewise linearization for distribution three-phase stochastic power flow," *IEEE Trans. on Smart Grid*, vol. 13, no. 2, pp. 1035–1048, 2022.

[8] Y. Chen, C. Wu, and J. Qi, "Data-driven Power Flow Method Based on Exact Linear Regression Equations," *Journal of Modern Power Systems and Clean Energy*, vol. 10, no. 3, pp. 800–804, 5 2022.

[9] Y. Liu, Y. Wang, N. Zhang, D. Lu, and C. Kang, "A data-driven approach to linearize power flow equations considering measurement noise," *IEEE Trans. on Smart Grid*, vol. 11, no. 3, pp. 2576–2587, 2020.

[10] S. Ramasamy, K. Ganesan, and V. Arunachalam, "Power Flow Parameter Estimation in Power System Using Machine Learning Techniques Under Varying Load Conditions," *International Journal of Electrical and Electronics Research*, vol. 10, no. 4, pp. 1299–1305, 2022.

[11] X. Hu, H. Hu, S. Verma, and Z.-L. Zhang, "Physics-guided deep neural networks for power flow analysis," *IEEE Trans. Power Syst.*, vol. 36, no. 3, pp. 2082–2092, 2021.

[12] Y. Yang, Z. Yang, J. Yu, B. Zhang, Y. Zhang, and H. Yu, "Fast Calculation of Probabilistic Power Flow: A Model-Based Deep Learning Approach," *IEEE Trans. on Smart Grid*, vol. 11, no. 3, pp. 2235–2244, 5 2020.

[13] H. Li, Y. Weng, V. Vittal, and E. Blasch, "Distribution grid topology and parameter estimation using deep-shallow neural network with physical consistency," *IEEE Trans. on Smart Grid*, pp. 1–1, 2023.

[14] Q. Yang, A. Sadeghi, G. Wang, G. B. Giannakis, and J. Sun, "Power system state estimation using gauss-newton unrolled neural networks with trainable priors," *2020 IEEE SmartGridComm*, pp. 1–6, 2020.

[15] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.

[16] W. Liao, B. Bak-Jensen, J. R. Pillai, Y. Wang, and Y. Wang, "A review of graph neural networks and their applications in power systems," *J. Mod. Power. Syst. Clean Energy*, vol. 10, no. 2, pp. 345–360, 2022.

[17] T. Wu, I. L. Carreno, A. Scaglione, and D. Arnold, "Spatio-Temporal Graph Convolutional Neural Networks for Physics-Aware Grid Learning Algorithms," *IEEE Trans. on Smart Grid*, 2023.

[18] B. Habib, E. Isufi, W. v. Breda, A. Jongepier, and J. L. Cremer, "Deep statistical solver for distribution system state estimation," *IEEE Trans. Power Syst.*, pp. 1–12, 2023.

[19] L. Pagnier and M. Chertkov. Physics-Informed Graphical Neural Network for Parameter and State Estimations in Power Systems.

[20] B. Donon, R. Clément, B. Donnot, A. Marot, I. Guyon, and M. Schoenauer, "Neural networks for power flow: Graph neural solver," *Electric Power Systems Research*, vol. 189, p. 106547, 2020.

[21] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural Message Passing for Quantum Chemistry," in *International Conference on Machine Learning*. PMLR, Jul. 2017, pp. 1263–1272.

[22] J. Du, S. Zhang, G. Wu, J. M. Moura, and S. Kar, "Topology adaptive graph convolutional networks," *arXiv preprint arXiv:1710.10370*, 2017.

[23] V. M. da Costa, N. Martins, and J. L. R. Pereira, "Developments in the newton raphson power flow formulation based on current injections," *IEEE Trans. Power Syst.*, vol. 14, no. 4, pp. 1320–1326, 1999.

[24] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2017.

[25] C. Josz, S. Fliscounakis, J. Maeght, and P. Panciatici, "Ac power flow data in matpower and qcqp format: itesla, rte snapshots, and pegase," *arXiv: Optimization and Control*, 2016.

[26] L. Thurner, A. Scheidler, F. Schäfer, J. Menke, J. Dollichon, F. Meier, S. Meinecke, and M. Braun, "pandapower — an open-source python tool for convenient modeling, analysis, and optimization of electric power systems," *IEEE Trans. Power Syst.*, vol. 33, no. 6, pp. 6510–6521, 11 2018.

[27] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2017.

[28] L. N. Smith and N. Topin, "Super-convergence: very fast training of neural networks using large learning rates," in *Defense + Commercial Sensing*, 2017.

[29] D. E. Hilt, D. W. Seegrist, U. S. F. Service., and N. F. E. Station, *Ridge, a computer program for calculating ridge regression estimates*. Upper Darby, Pa, Dept. of Agriculture, Forest Service, Northeastern Forest Experiment Station, 1977, vol. no.236.

如观察所示，PowerFlowNet 的所有变体均未达到完整模型的性能。在单层模型中，可以明显看出该模型无法捕捉远节点依赖关系，导致误差高达 55 倍。类似地，在不传递消息的情况下，由于完全忽略了边特征，该模型的性能也很差（误差高达 390 倍），这凸显了引入边特征的重要性。另一方面，不传递消息的单层模型性能最佳（误差高达 40 倍），因为它的功能类似于堆叠式多层感知器 (MLP)。因此，该分析表明，PowerFlowNet 的每个部分对于实现高质量的 PF 近似都至关重要。

副主席

本文提出了一种新型数据驱动算法 PowerFlowNet。该算法充分利用了图神经网络（GNN）运算的高效性和强大功能，并将其应用于电力网络拓扑结构，从而能够精确逼近功率因数（PF）。具体而言，我们的方法将传统的功率因数问题转化为 GNN 节点回归任务，将母线表示为节点，输电线路表示为边，同时保持网络连通性。我们的模型采用了一种独特的配置，结合了掩码编码器和一系列我们提出的 PowerFlowConv 层，旨在聚合整个图的特征，并自然地学习底层功率因数的动态特性。实验结果验证了 PowerFlowNet 能够为各种网络案例生成高质量的解决方案，其中包括法国 6470rte 超大规模电力网络。更重要的是，无论网络规模如何，我们的方法都能在远低于传统求解器的时间下生成精确的功率因数预测结果，在执行时间方面优于成熟的牛顿-拉夫逊法，同时结果也与之非常接近。最终，我们的消融研究揭示了我们方法的稳健性和潜在局限性，从而使 PowerFlowNet 能够在各种现实世界的电力系统运行和规划场景中灵活部署。

R

[1] M. Albadi，“潮流分析”，载于《工程计算模型》，K. Volkov 主编，里耶卡：IntechOpen，2019 年，第 5 章。[2] VM da Costa、N. Martins 和 JLR Pereira，“基于电流注入的牛顿-拉夫逊潮流公式的发展”，

IEEE 电力系统学报，第 14 卷，第 1320-1326 页，1999 年。

[3] A. Eltamaly 和 AA Elghaffar，“高斯-赛德尔法潮流分析”
方法；调查”，《国际机电一体化、电气和计算机技术杂志》(IJMEC) 2305-0543, 2411-6173, 第 x 卷, 2017 年 10 月。

[4] H. Seifi 和 M. Sepasian，《电力系统规划：问题、算法和解决方案》。Springer，2011 年 1 月，第 49 卷。

[5] Y. Liu, N. Zhang, Y. Wang, J. Yang 和 C. Kang，“数据驱动的潮流线性化：一种回归方法”，IEEE 智能电网汇刊，第 10 卷，第 3 期，第 2569–2580 页，2019 年。

[6] L. Guo, Y. Zhang, X. Li, Z. Wang, Y. Liu, L. Bai 和 C. Wang，“数据驱动的潮流计算方法：一种提升维数线性回归方法”，IEEE 电力系统汇刊，第 37 卷，第 3 期，第 1-10 页，2019 年。

1798–1808，5 2022。

[7] J. Chen、W. Wu 和 LA Roald，“基于数据的分段线性化方法用于配电三相随机潮流计算”，IEEE 智能电网汇刊，第 13 卷，第 2 期，第 1035–1048 页，2022 年。

[8] Y. Chen、C. Wu 和 J. Qi，“基于精确线性回归方程的数据驱动潮流计算方法”，现代电力系统与清洁能源杂志，第 10 卷，第 3 期，第 800–804 页，2022 年。

[9] Y. Liu, Y. Wang, N. Zhang, D. Lu 和 C. Kang，“考虑测量噪声的线性化潮流方程的数据驱动方法”，IEEE 智能电网汇刊，第 11 卷，第 3 期，第 2576-2587 页，2020 年。

[10] S. Ramasamy、K. Ganesan 和 V. Arunachalam，“潮流参数
“在变化的负载条件下使用机器学习技术对电力系统进行估计”，《国际电气与电子研究杂志》，第 10 卷，第 4 期，第 1299-1305 页，2022 年。

[11] X. Hu、H. Hu、S. Verma 和 Z.-L. Zhang，“物理引导的深度神经网络
用于潮流分析的网络”，IEEE 电力系统汇刊，第 36 卷，第 3 期，第 2082-2092 页，2021 年。

[12] Y. Yang，Z. Yang，J. Yu，B.Zhang，Y.Zhang，and H.Yu，“快速
概率潮流计算：基于模型的深度学习方法”，IEEE 智能电网汇刊，第 11 卷，第 3 期，第 2235-2244 页，2020 年 5 月。

[13] H. Li、Y. Weng、V. Vittal 和 E. Blasch，“配电网拓扑和
使用具有物理一致性的深浅神经网络进行参数估计”，IEEE 智能电网汇刊，第 1-1 页，2023 年。

[14] Q. Yang、A. Sadeghi、G. Wang、GB Giannakis 和 J. Sun，《权力》
使用具有可训练先验的高斯-牛顿展开神经网络进行系统状态估计”，2020 年 IEEE SmartGridComm，第 1-6 页，2020 年。

[15] TN Kipf 和 M. Welling，“基于图的半监督分类”
卷积网络”，第五届国际学习表征会议，ICLR 2017，法国土伦，2017 年 4 月 24-26 日，会议论文集。OpenReview.net，2017。

[16] W. Liao、B. Bak-Jensen、JR Pillai、Y. Wang 和 Y. Wang，“图神经网络及其在电力系统中的应用综述”，J.
现代动力系统与清洁能源，第 10 卷，第 2 期，第 345-360 页，2022 年。

[17] T. Wu、IL Carreno、A. Scaglione 和 D. Arnold，“时空
“面向物理感知电网学习算法的图卷积神经网络”，IEEE 智能电网汇刊，2023 年。

[18] B. Habib、E. Isufi、W. v. Breda、A. Jongepier 和 JL Cremer，“用于配电系统状态估计的深度统计求解器”，IEEE Trans.
电力系统，第 1-12 页，2023 年。

[19] L. Pagnier 和 M. Chertkov. 基于物理信息的图形神经网络用于电力系统参数和状态估计。[20] B. Donon、R. Cl´ement、B. Donnot、A. Marot、I. Guyon 和 M. Schoe-
Nauer，“用于潮流计算的神经网络：图神经网络求解器”，电力系统研究，第 189 卷，第 106547 页，2020 年。

[21] J. Gilmer、SS Schoenholz、PF Riley、O. Vinyals 和 GE Dahl，
“用于量子化学的神经信息传递”，国际机器学习会议论文集。PMLR，2017 年 7 月，第 1263-1272 页。

[22] J. Du、S. Zhang、G. Wu、JM Moura 和 S. Kar，“拓扑自适应图卷积网络”，arXiv 预印本 arXiv:1710.10370，2017 年。[23] VM da Costa、N. Martins 和 JLR Pereira，“基于电流注入的牛顿-拉夫逊潮流公式的发展”，

IEEE 电力系统学报，第 14 卷，第 4 期，第 1320-1326 页，1999 年。

[24] A. Vaswani、N. Shazeer、N. Parmar、J. Uszkoreit、L. Jones、AN Gomez、L. Kaiser 和 I. Polosukhin，“注意力就是您所需要的”，2017 年。 [25] C. Josz、S. Fliscounakis、J. Maeght 和 P. Panciatici，“matpower 和 qcqp 格式的交流潮流数据： itesla、rte snapshots 和 pegase，”
arXiv：优化与控制，2016。

[26] L.瑟纳、A.谢德勒、F.谢弗、J.门克、J.多利雄、F.迈耶、
S. Meinecke 和 M. Braun，“pandapower — 一个用于方便地对电力系统进行建模、分析和优化的开源 Python 工具”，IEEE 电力系统汇刊，第 33 卷，第 6 期，第 6510-6521 页，2018 年 11 月。

[27] I. Loshchilov 和 F. Hutter，“解耦权重衰减正则化”，
在 2017 年国际学习表征会议上。

[28] LN Smith 和 N. Topin，“超收敛：极快的训练
“使用大学习率的神经网络”，发表于《国防与商业传感》，2017 年。

[29] DE Hilt、DW Seegrist、USF Service 和 NFE Station，Ridge，
用于计算岭回归估计的计算机程序。宾夕法尼亚州上达比，农业部林务局东北森林实验站，1977 年，第 236 卷。